

Redes Neuronales

Módulo 4 - parte 2

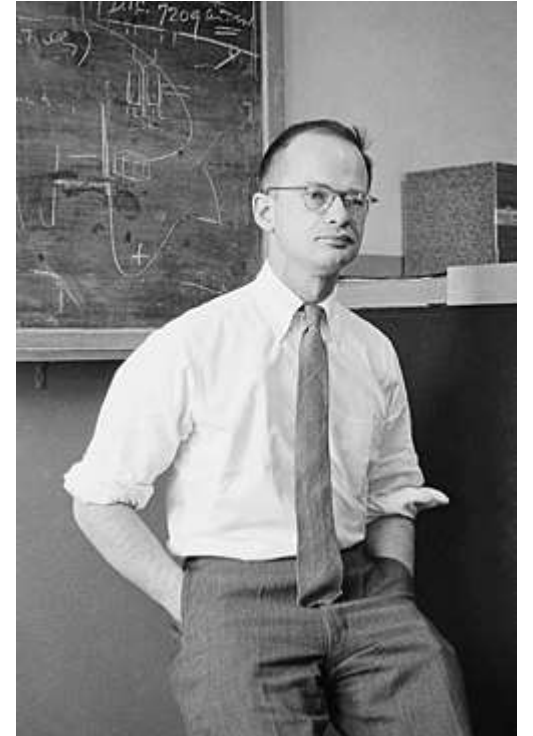
Temario

- La Neurona Artificial
- El Perceptrón
- El Perceptrón Multicapa
- Las Redes Neuronales y la Retropropagación
 - El Descenso del Gradiente

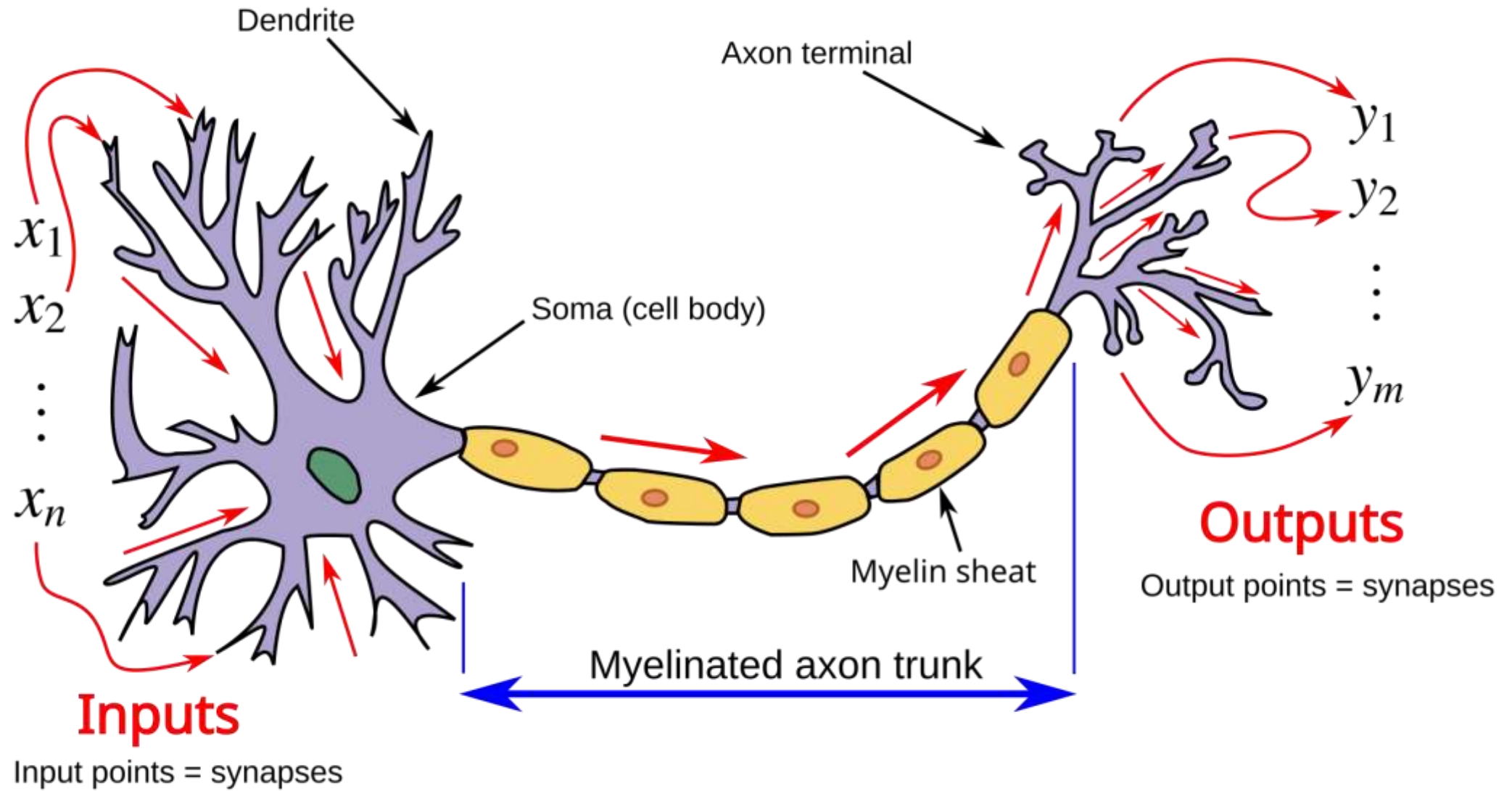
La Neurona Artificial

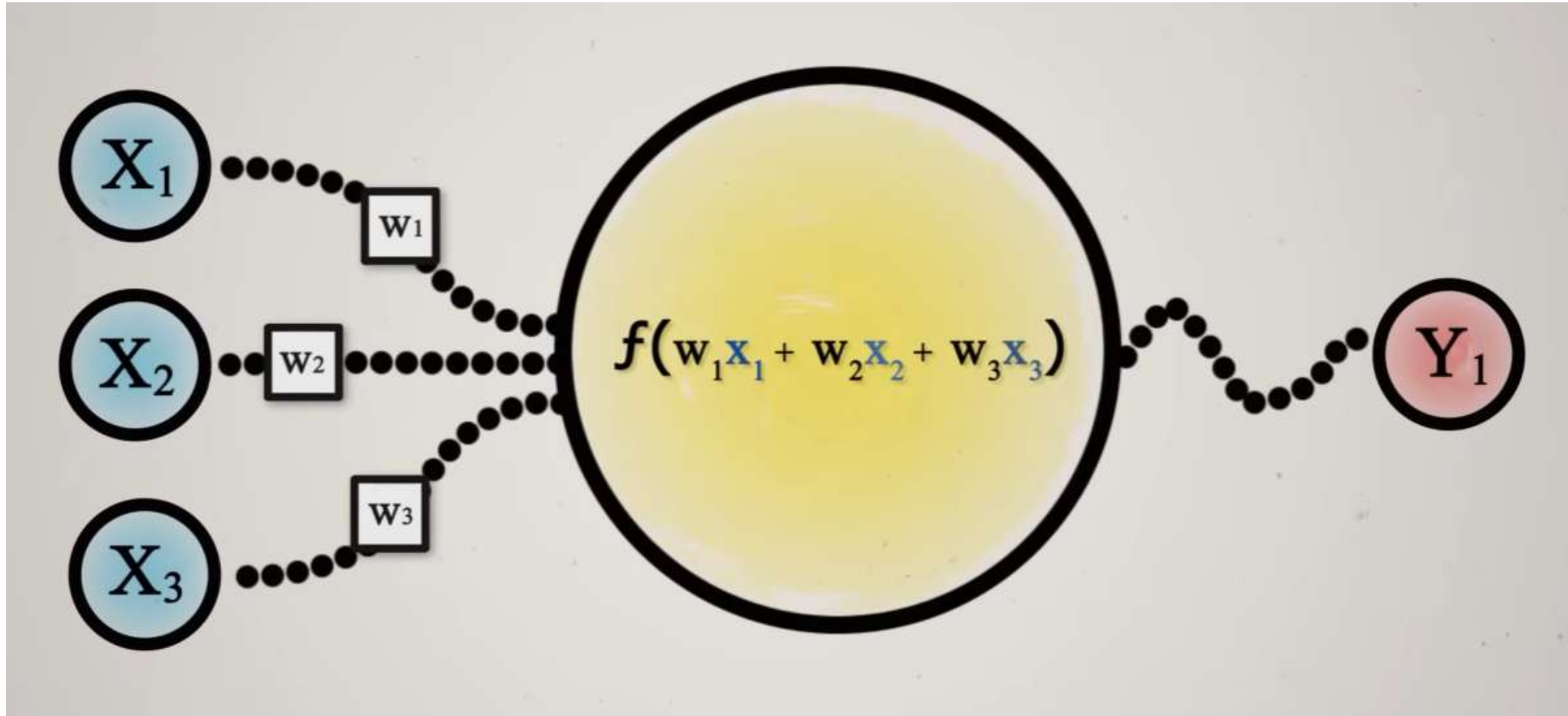
La primera neurona artificial

- 1943 – McCulloch y Pitts
- Es un **clasificador lineal binario** capaz de separar datos en **dos categorías** si estos son **linealmente separables**
- Características:
 - Entradas
 - Pesos (fijos) **No había aprendizaje**
 - Salida
 - Función de activación: escalón
- Podía emular las lógica AND, OR entre otras
- Fue un modelo teórico



Pitts en el MIT





¿Cómo funciona?

Funciona tomando múltiples **entradas**, multiplicándolas por sus respectivos "**pesos**" para asignarles importancia, sumando estos productos y luego aplicando una **función de activación** (con un umbral) para producir una **salida** binaria, usualmente un "1" o un "0".

Las matemáticas de la Neurona Artificial

Suma ponderada:

$$z = \sum_{i=1}^n w_i x_i$$

x = entradas

w = pesos

Función de activación:

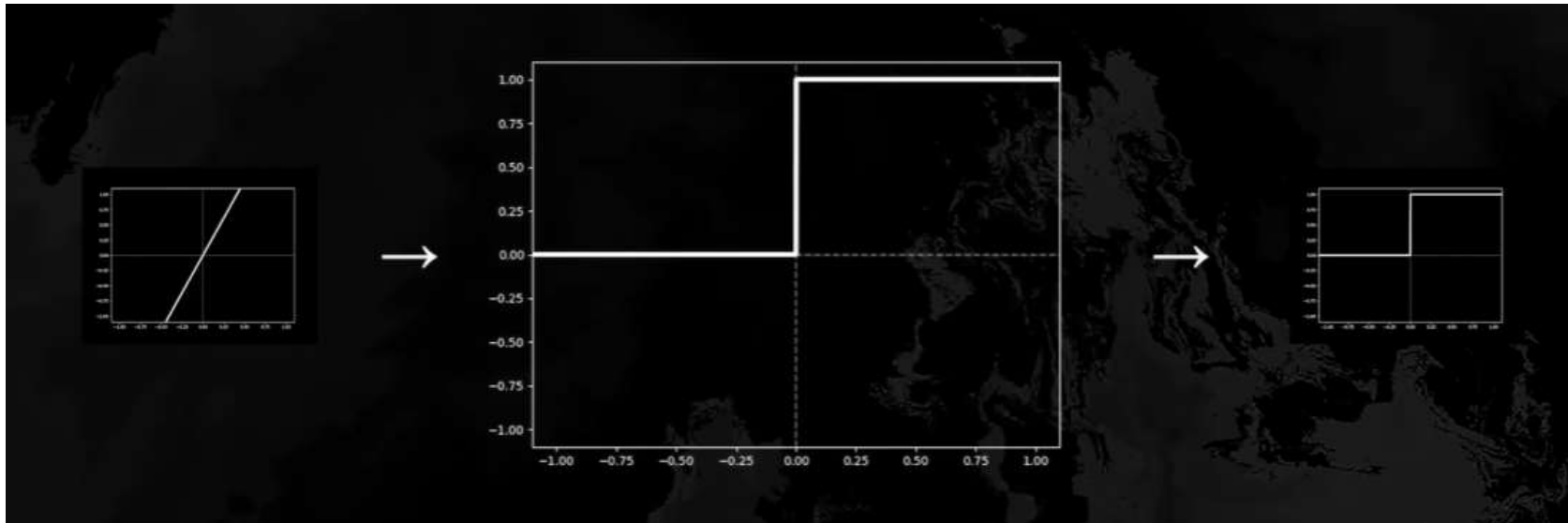
$$f(z) = \begin{cases} 1 & \text{si } z \geq \alpha \\ 0 & \text{si } z < \alpha \end{cases}$$

α = umbral que puede variar,
generalmente es 0

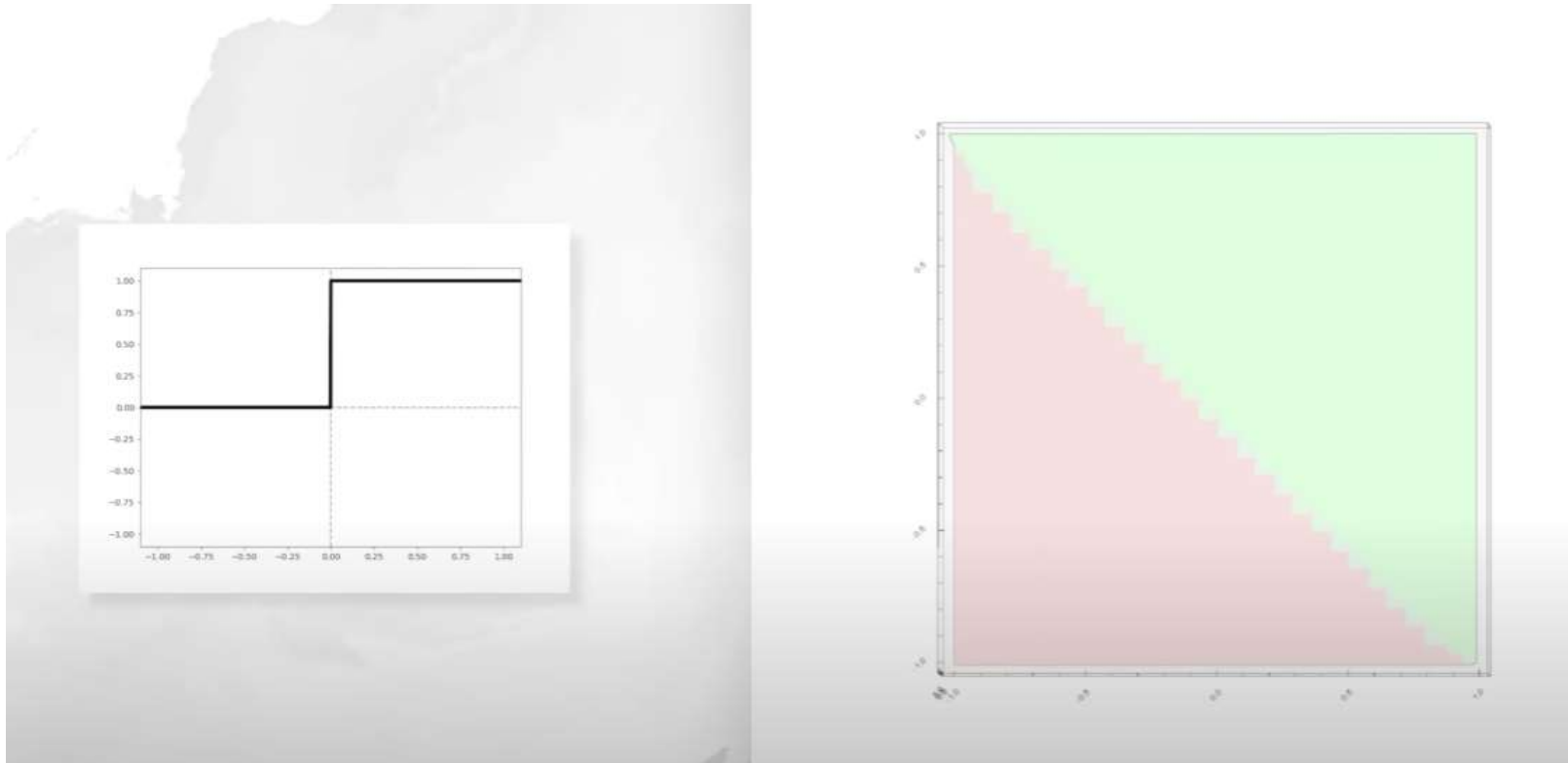
Salida:

$$y = f(z)$$

La función de activación: Escalonada

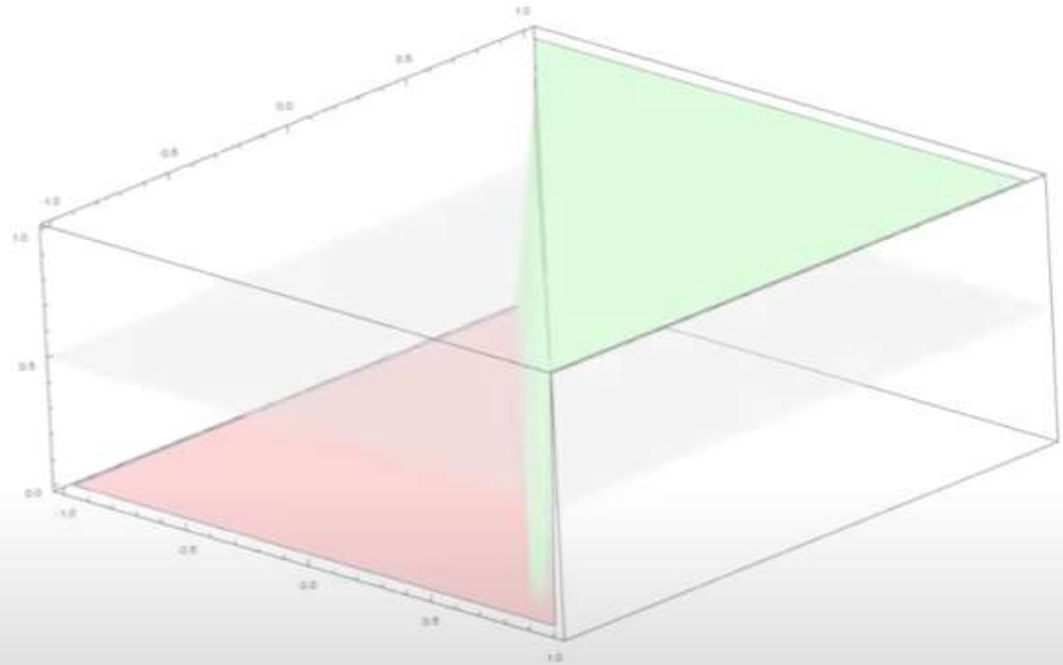
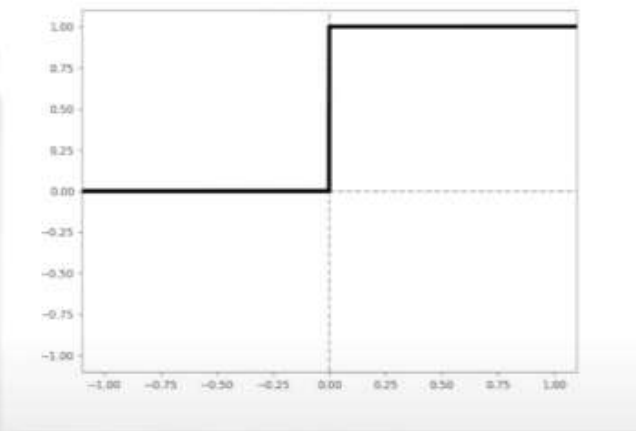


La función de activación: Escalonada



La función de activación distorciona el plano generado por la neurona.

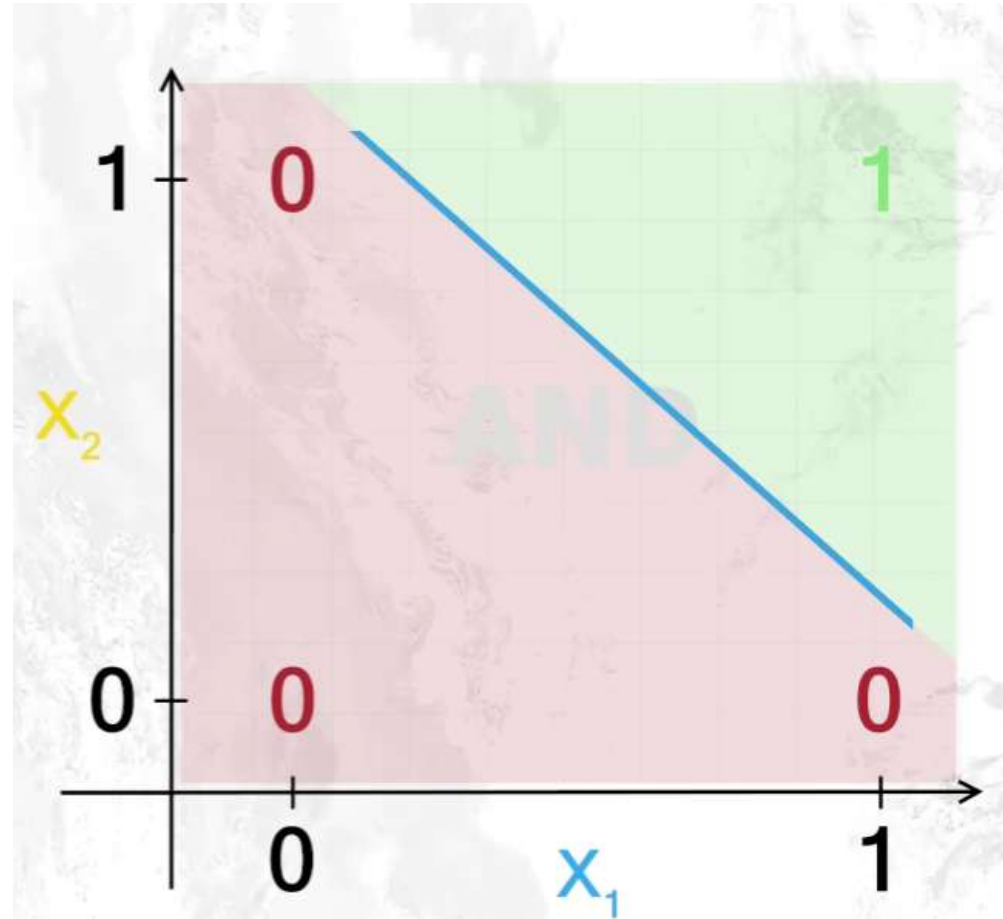
La función de activación: Escalonada



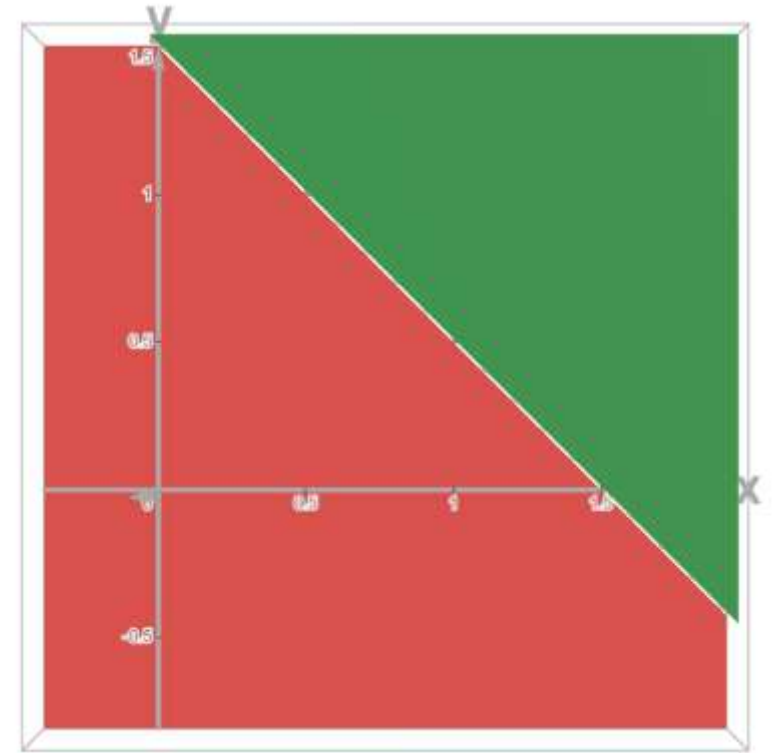
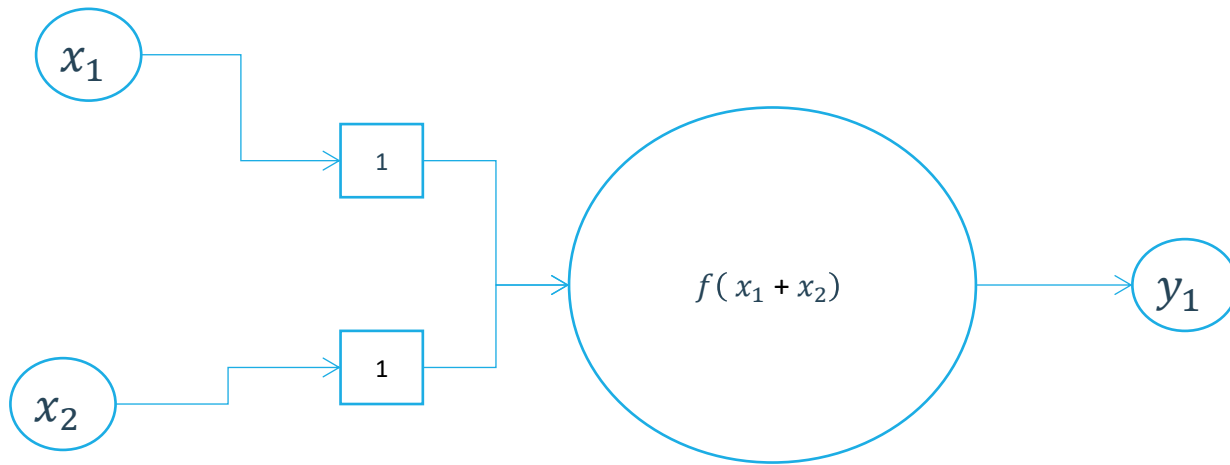
La función de activación distorciona el plano generado por la neurona.

Ejemplo AND

a	b	AND
0	0	0
0	1	0
1	0	0
1	1	1



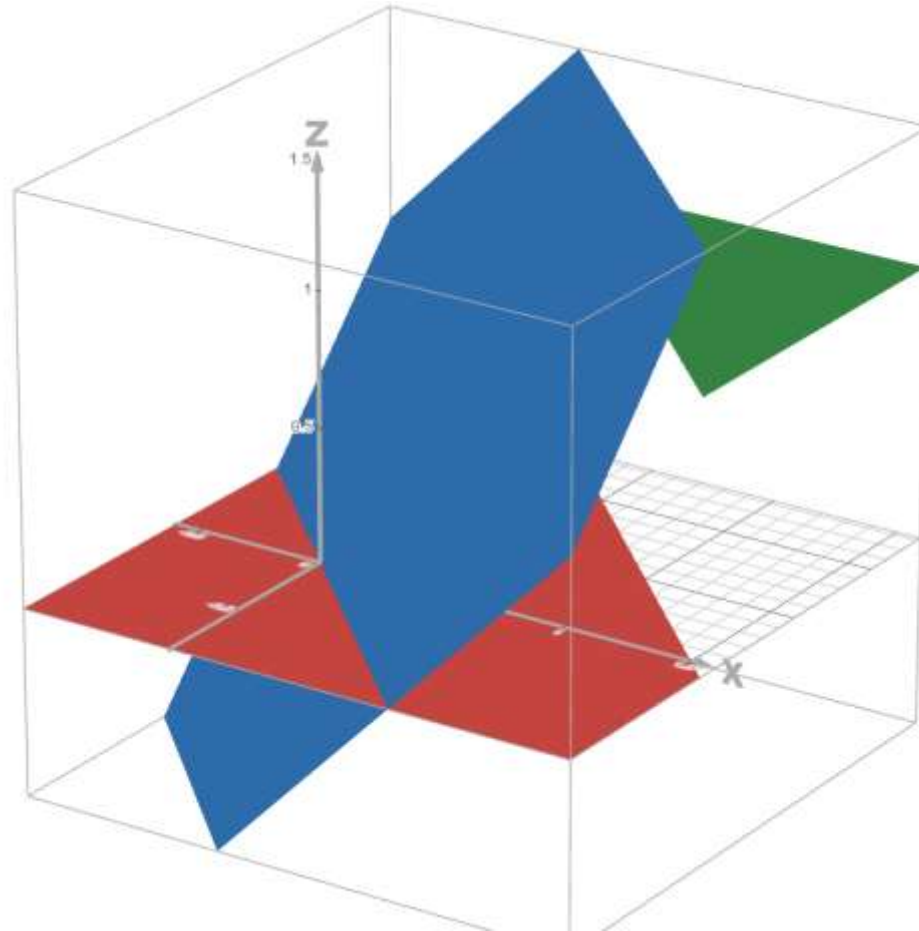
Ejemplo AND



x_1	x_2	w_1	w_2	$z = \text{Suma}$	$y = f(z)$	Salida deseada	Parámetros:	
0	0	0	1	1	0	0	Umbral	1,5
1	0	0			1	0		
0	1	1			1	0		
1	1	1			2	1		

El ejemplo en 3D

- | | | |
|---|---|------------------------------|
| 1 |  | $z = x + y$ |
| 2 |  | $z = \{x + y \geq 1.5 : 1\}$ |
| 3 |  | $z = \{x + y < 1.5 : 0\}$ |



<https://www.desmos.com/3d/dbxk0mo3q1>

Ejercicios

- Con Excel crear una red neuronal que resuelva la compuerta lógica OR
- Crear código en Python para emular una neurona artificial

Limitaciones

- Entradas y salidas binarias
- Pesos fijos e iguales lo cual implica que no “aprende”
- Función de activación escalón
- Solo funciones linealmente separables

El Perceptrón

El Perceptrón

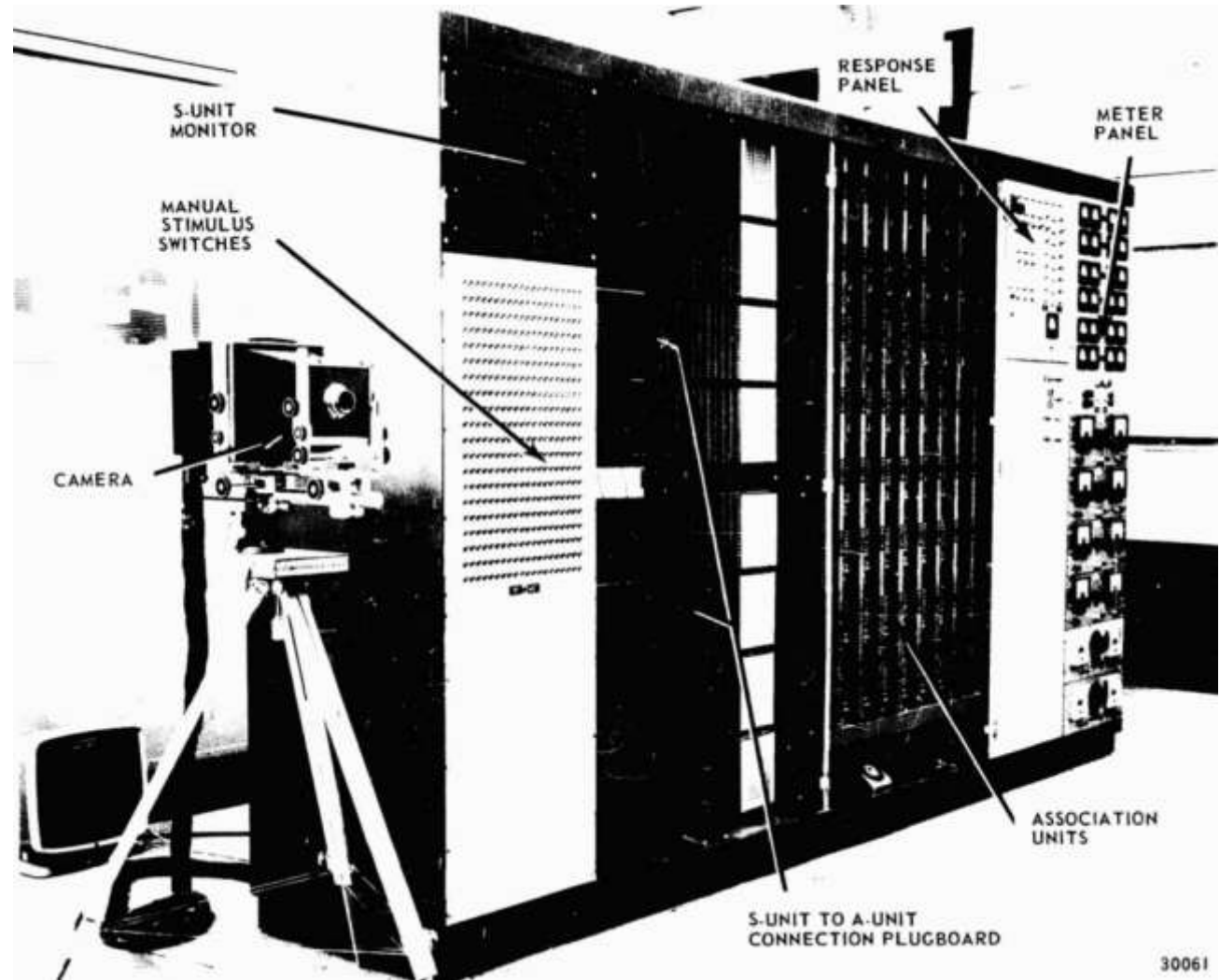
- 1957 – Frank Rosenblatt
- Es un clasificador lineal capaz de separar datos en dos categorías si estos son linealmente separables
- Características:
 - Una neurona
 - **Pesos que se aprenden con la "regla de aprendizaje"**
 - Función escalón o Sign function
- Reconocimiento de patrones (carácteres, clasificación)
- Clasificación lineal binaria inicialmente o también continua si la función de activación es la identidad (o sea, como si no estuviera)
- Se creó hardware para resolver problemas prácticos

El Perceptrón



Frank Rosenblatt

https://www.youtube.com/watch?v=cNxadbrN_al

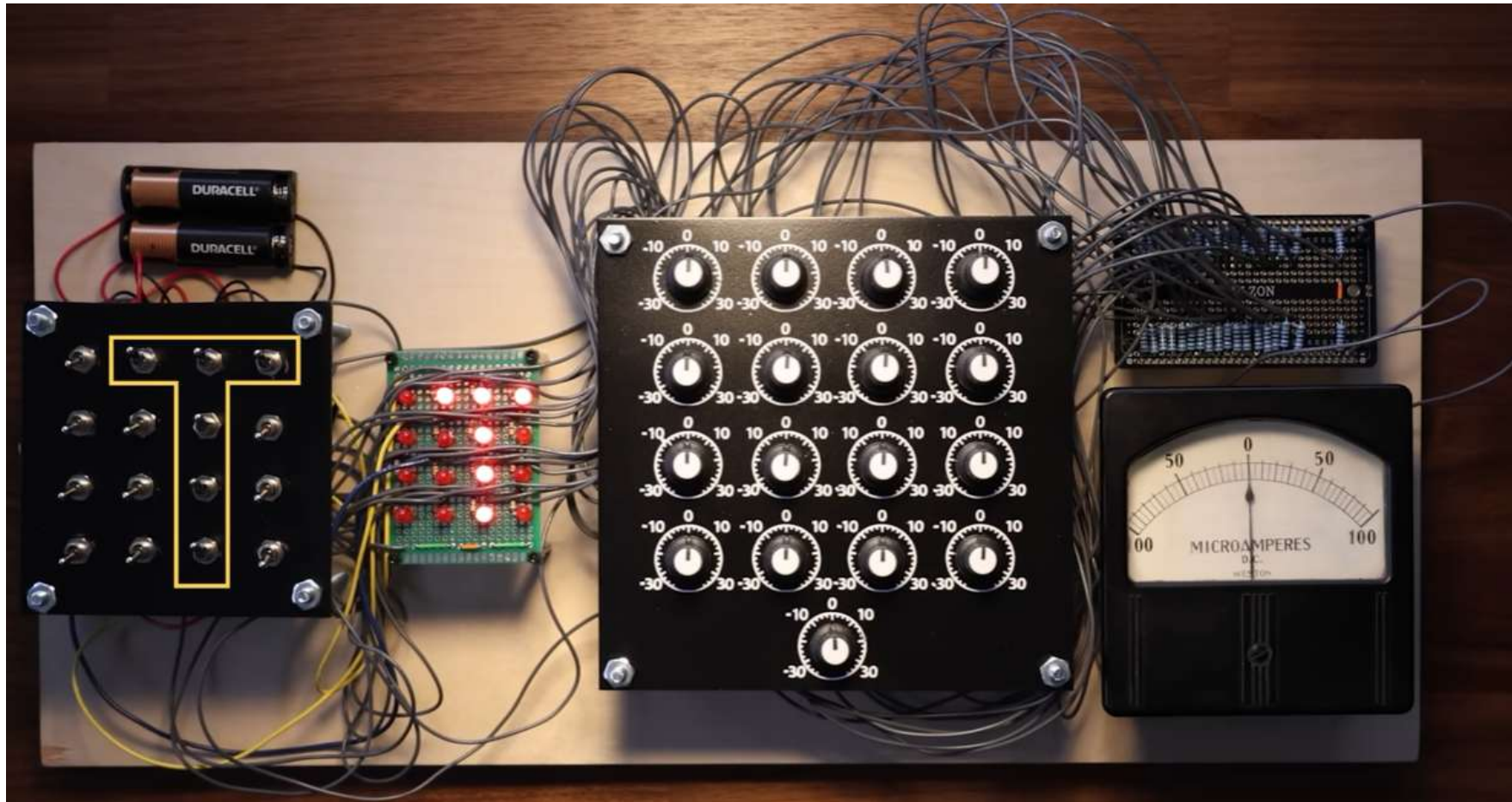


Mark I Perceptron

¿Cómo funciona?

Funciona tomando múltiples **entradas**, multiplicándolas por sus respectivos "**pesos**" para asignarles importancia, sumando estos productos y luego aplicando una **función de activación** (con un umbral) para producir una **salida** binaria, usualmente un "sí" o un "no".

El Perceptrón



<https://www.youtube.com/watch?v=l-9ALe3U-Fg> hasta 2:54

Las matemáticas del Perceptrón

Suma ponderada:

$$z = \sum_{i=1}^n w_i x_i + b w_0$$

x = entradas

w = pesos

b = sesgo

Función de activación:

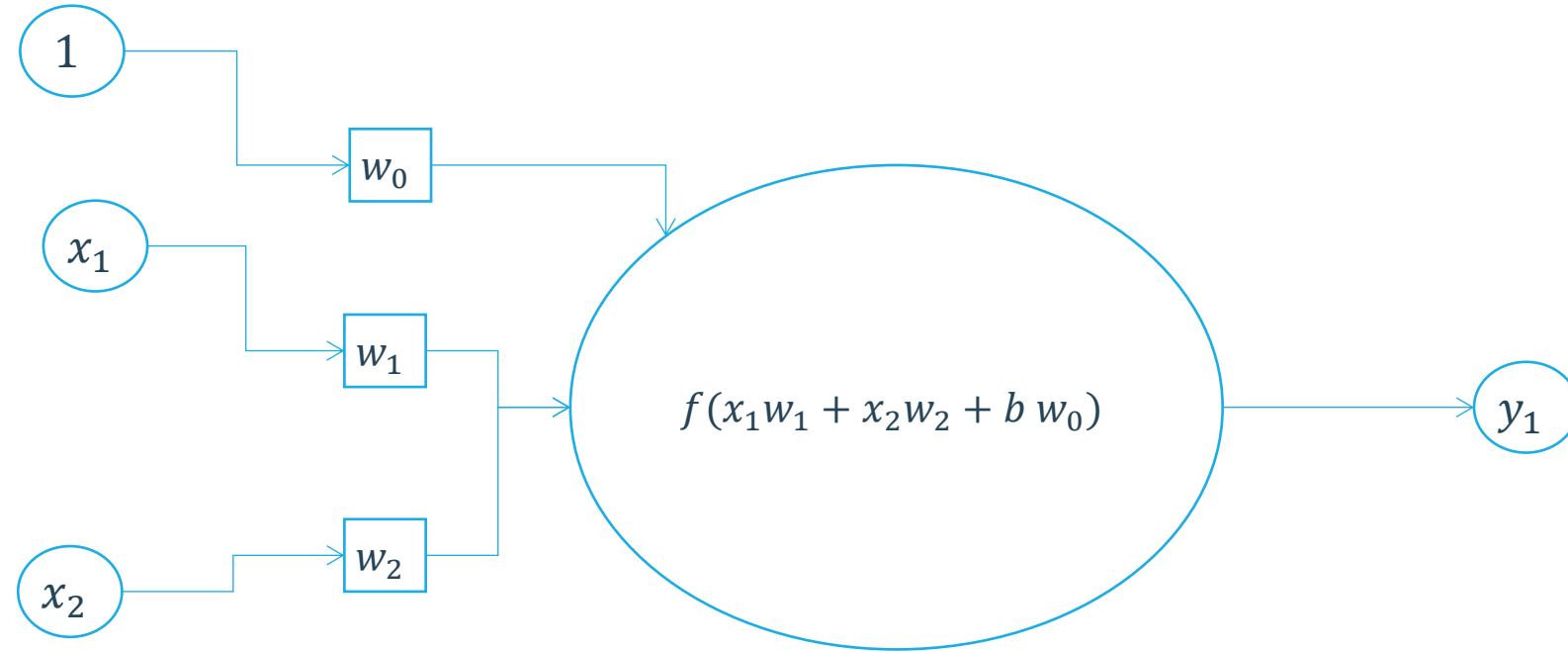
$$f(z) = \begin{cases} 1 & \text{si } z \geq \alpha \\ 0 & \text{si } z < \alpha \end{cases}$$

α = umbral que puede variar,
generalmente es 0

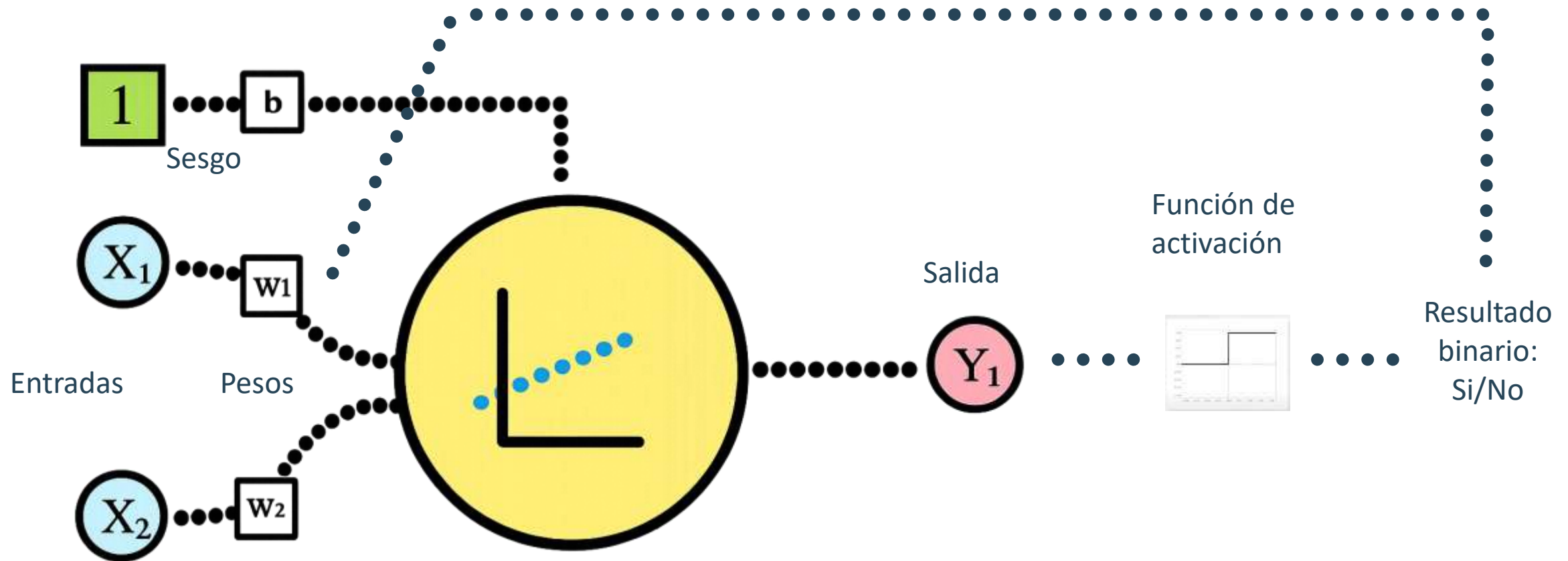
Salida:

$$y = f(z)$$

El Perceptrón



Si hay error, se actualizan los pesos



$$y = w_1 x_1 + w_2 x_2 + b$$

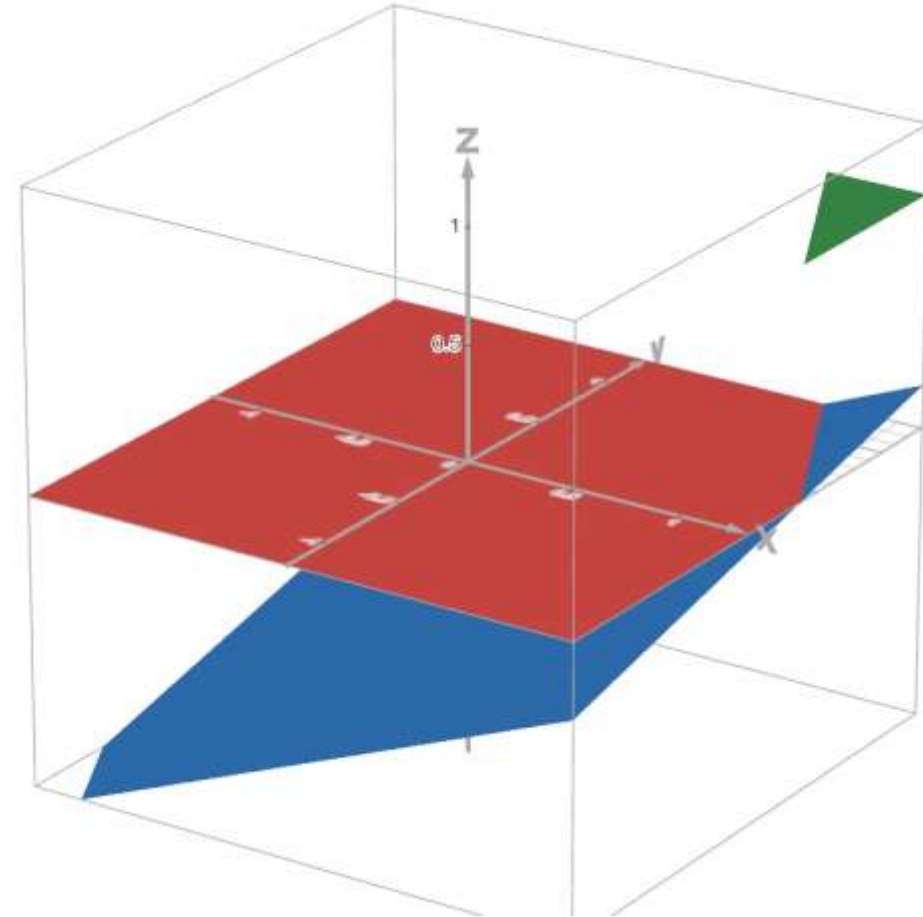
La regla de aprendizaje: el entrenamiento

1. **Inicializar:** Asignar pesos y sesgo aleatorios pequeños (o cero).
2. **Iterar:** Repetir hasta que clasifique correctamente **todas** las entradas del conjunto de entrenamiento:
 - a) Calcular la suma ponderada z
 - b) Calcular la salida del perceptrón $y=f(z)$ usando la función de activación.
 - c) Calcular el error $e = d - y$. (d es la salida deseada, y es la salida predicha)
 - d) Calcular la variación Δw que aplicaremos a los siguientes pesos:
 $\Delta w = \eta * e * x_i$ (η es la tasa de aprendizaje)
 - e) Ajustar los pesos:

$$w_{i \text{ nuevo}} = w_{i \text{ antiguo}} + \Delta w_i$$

El Perceptrón

1		$z = 0.4x + 0.2y - 0.6$
2		$z = \{ 0.4x + 0.2y - 0.6 \geq 0 : 1 \}$
3		$z = \{ 0.4x + 0.2y - 0.6 < 0 : 0 \}$



<https://www.desmos.com/3d/cgxxxyxbilp>

El Perceptrón en la práctica

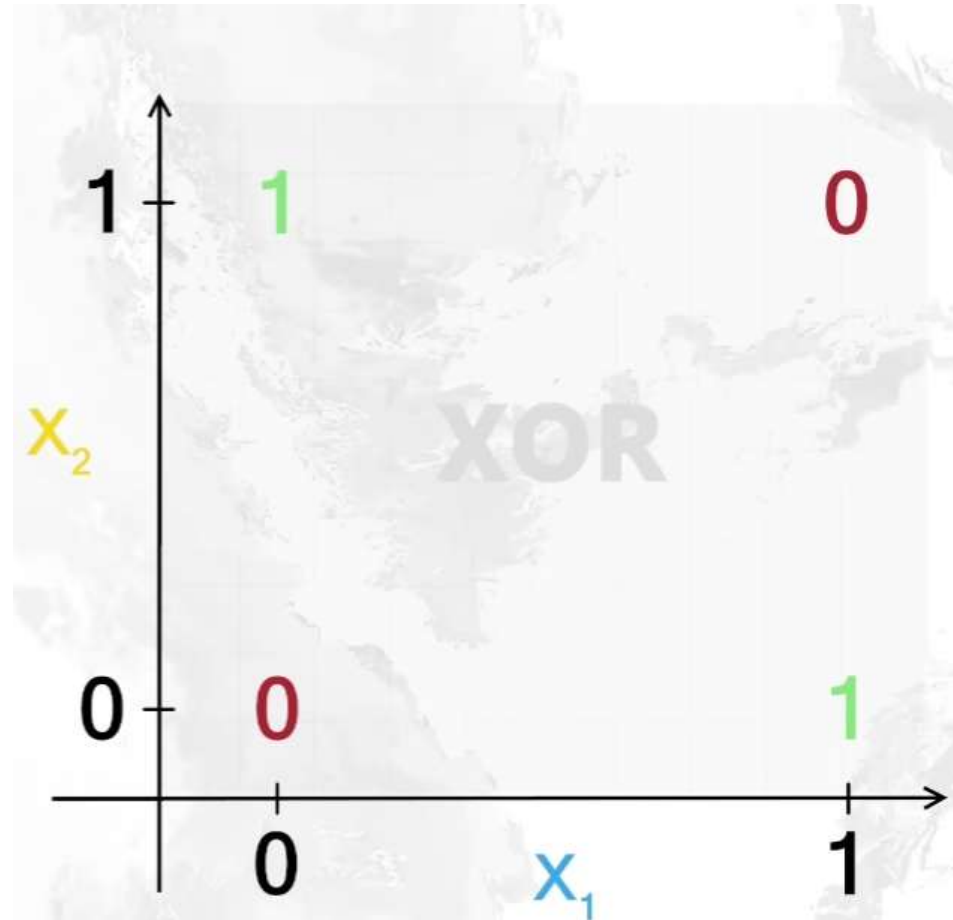
- Con Excel

Ejercicios

- Implementar el perceptrón en Python
- Hacer un Perceptrón que lea números de una matriz de 3 x 5 en Excel. Entrenarlo para que reconozca el número 7, añadiendo el 1,3,4 y 5 como negativos.
- Implementarlo también en Python

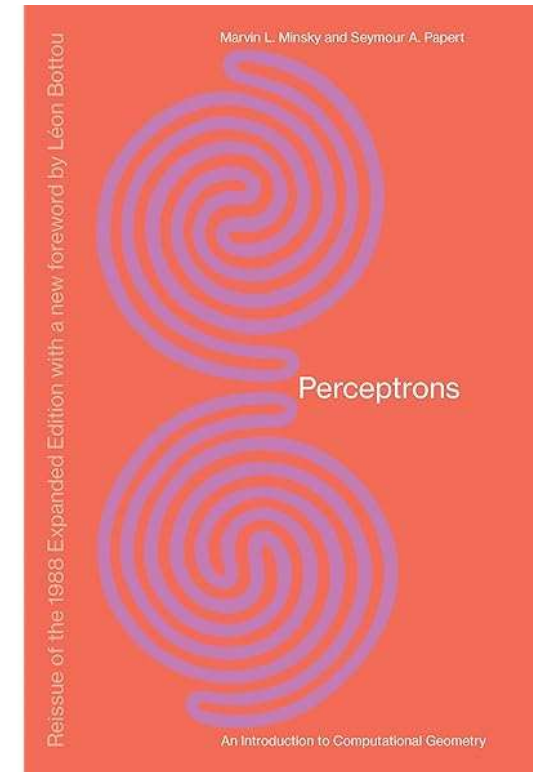


El problema de la compuerta lógica XOR



El problema del Perceptrón

- Con una sola neurona no se puede resolver una compuerta XOR
- Solo puede resolver problemas **linealmente** separables
- En 1969 se publica el libro “Perceptrons” dando inicio al primer “inverno de la IA”



El Perceptrón Multicapa

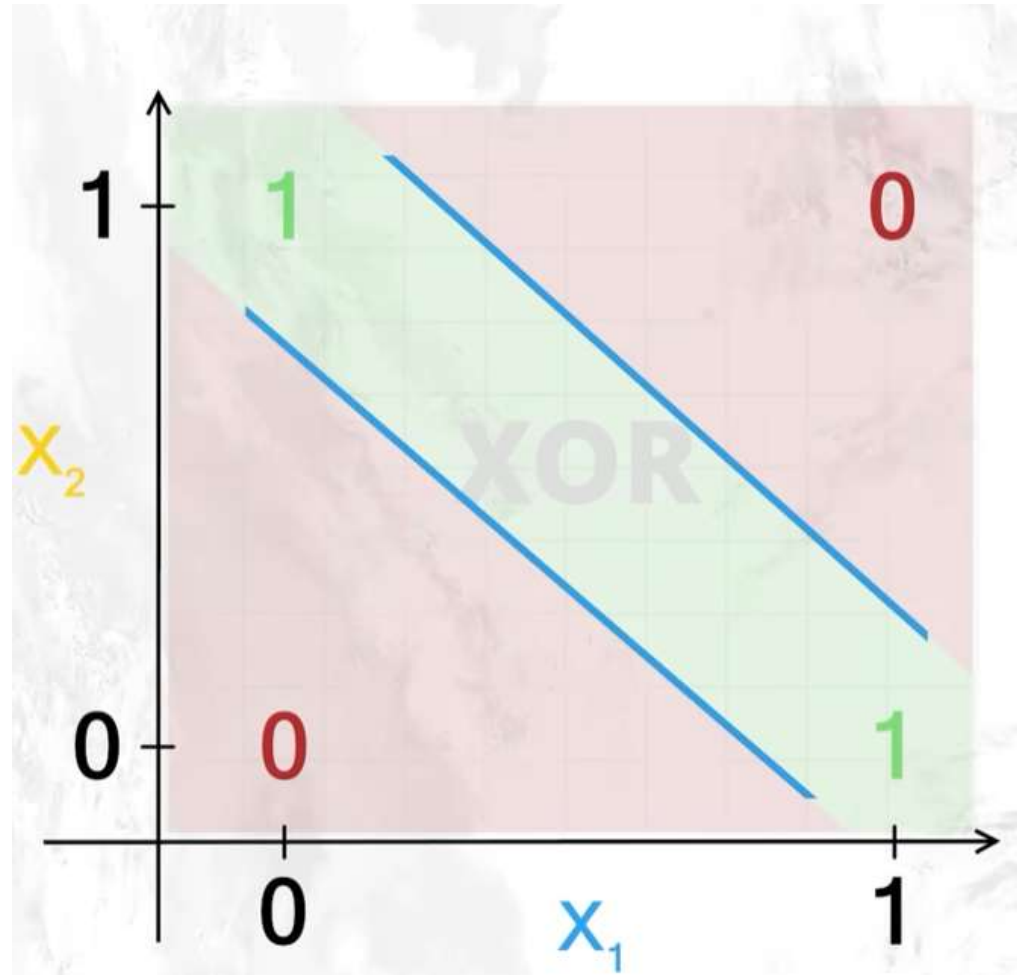
La solución al problema XOR

El Perceptrón Multicapa

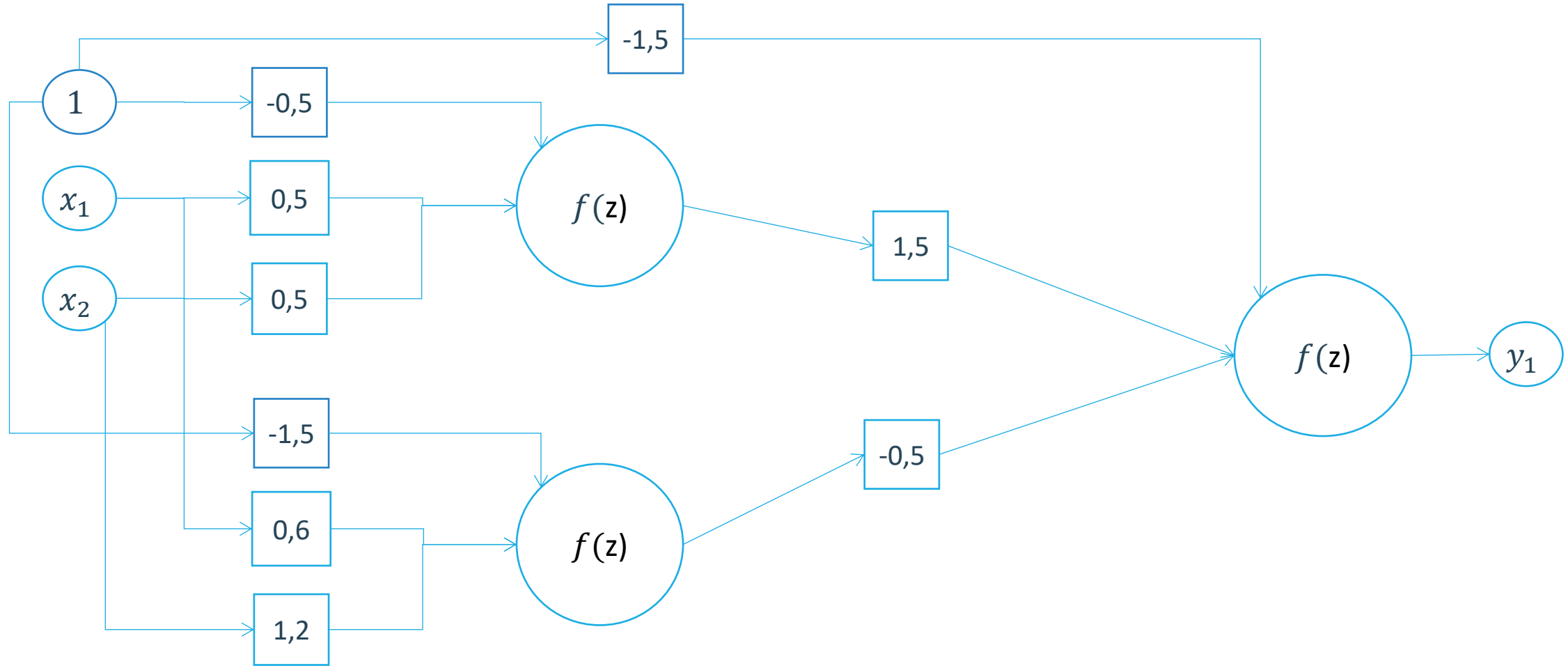
- Desarrollo: 1965 por Alexey Ivakhnenko
- Más neuronas y más de una capa permiten modelar funciones más complejas
- Más funciones de activación (Sigmoide, ReLu, etc.)



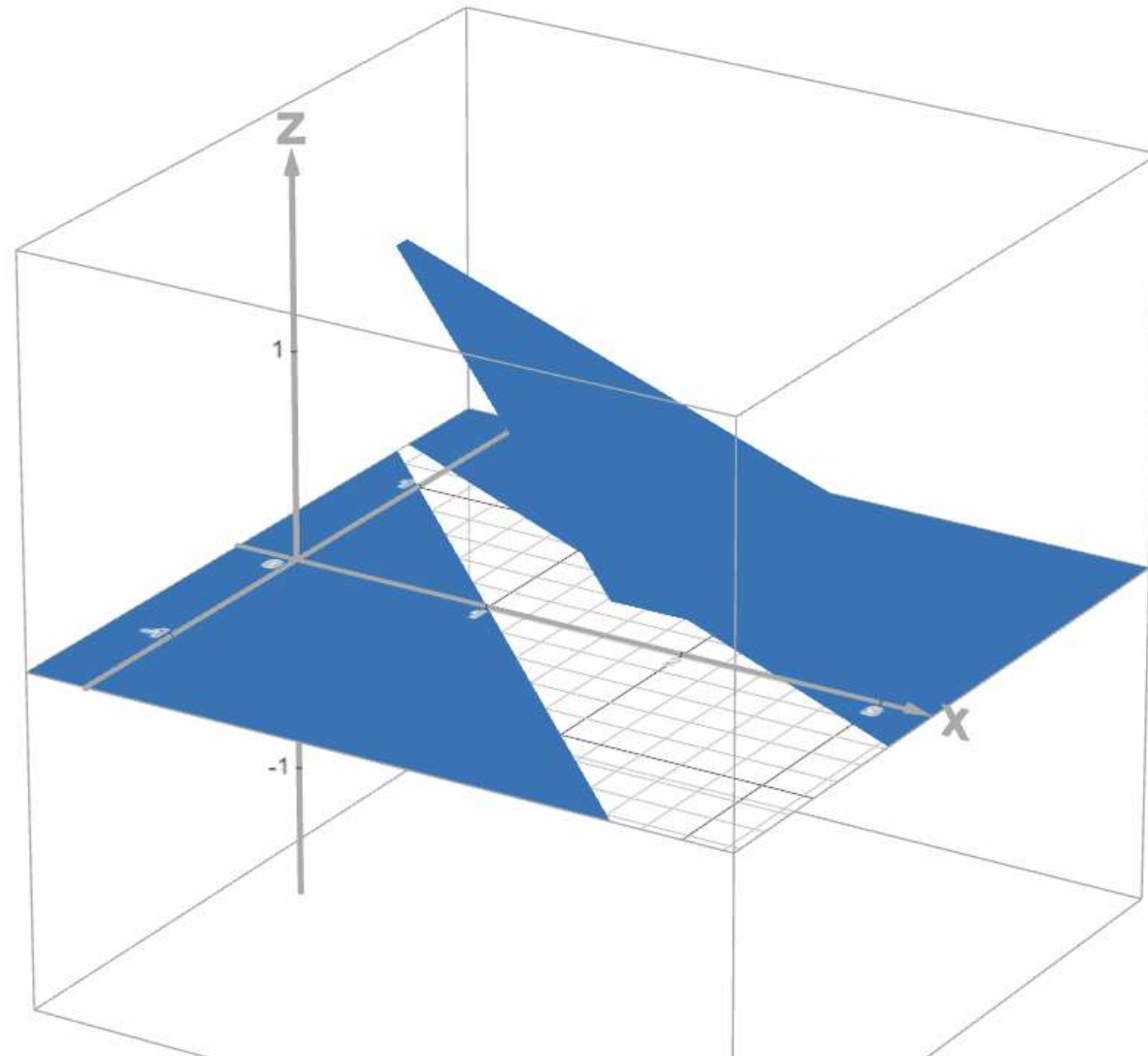
La solución



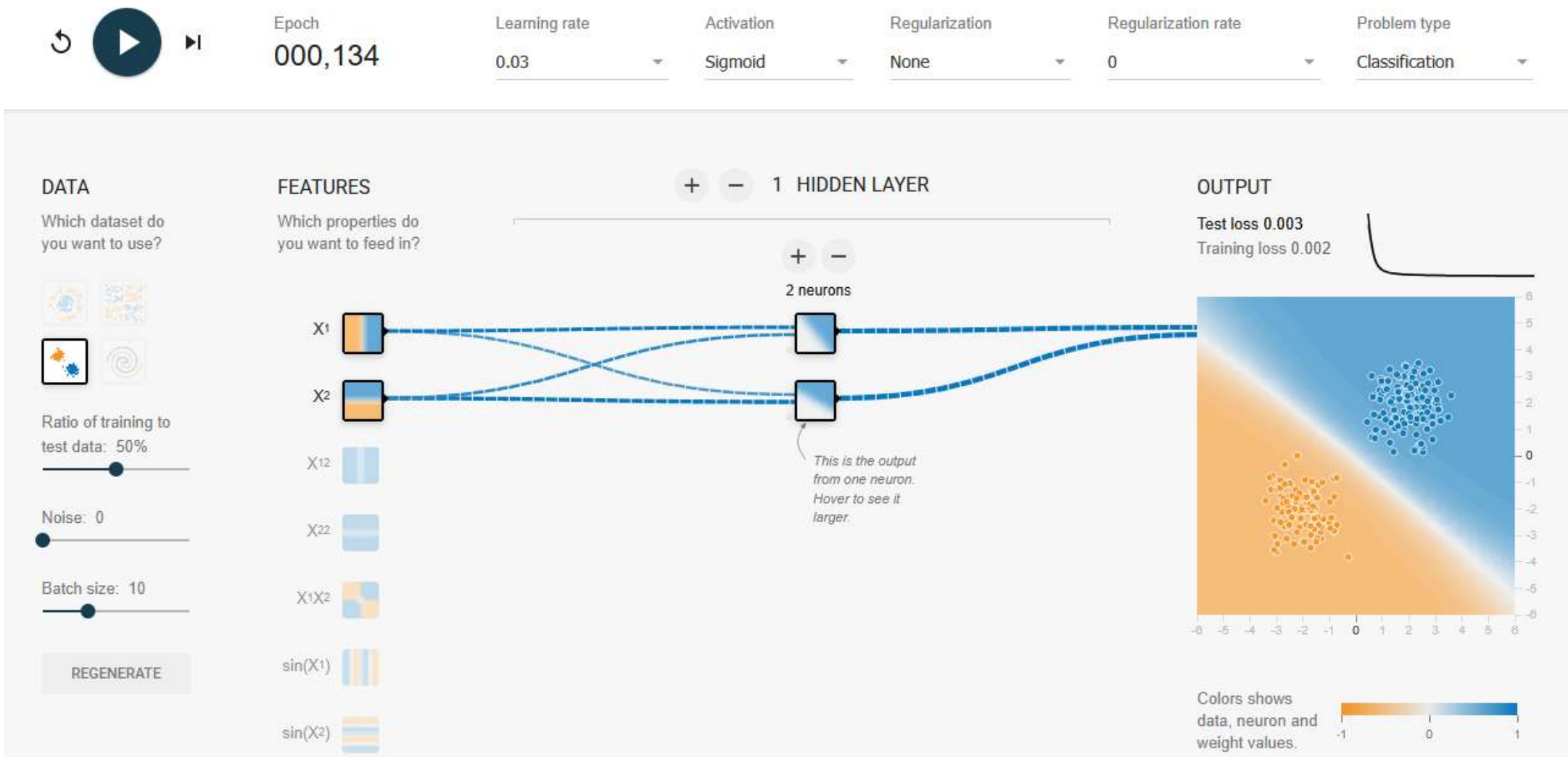
La solución usando Excel



La solución



<https://www.desmos.com/3d/hnzujhl1bb>

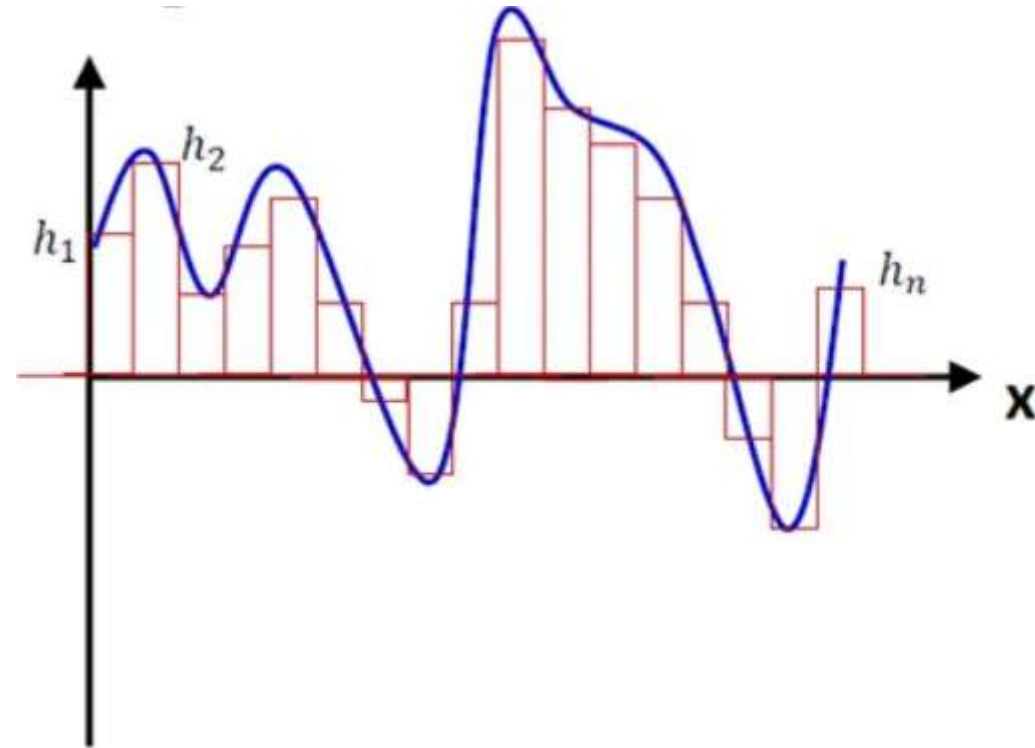


<https://playground.tensorflow.org>

Teorema de aproximación universal

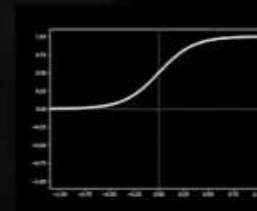
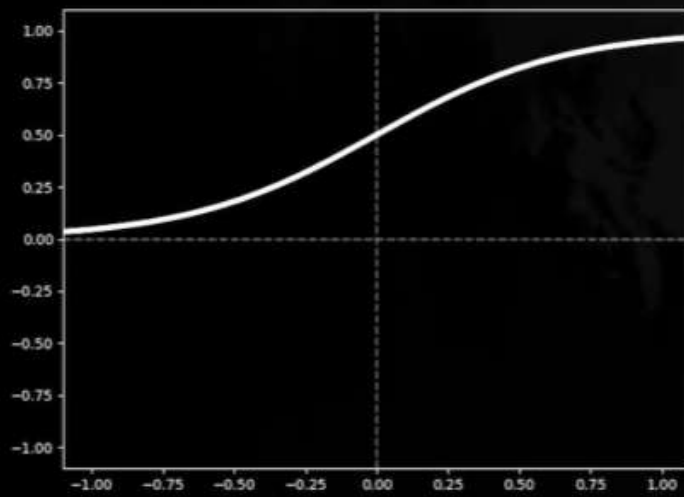
Una red neuronal de tipo "feedforward" (o alimentación hacia adelante) con al menos una capa oculta, y con un número finito de neuronas, puede **aproximar cualquier función continua** a un grado arbitrario de precisión, siempre que la función de activación utilizada en las neuronas de la capa oculta sea **no lineal** (sino sería como una sola) y **continua** (como la función sigmoide, ReLU, tanh, etc.).

Teorema de aproximación universal



FUNCION DE ACTIVACION

SIGMOIDE

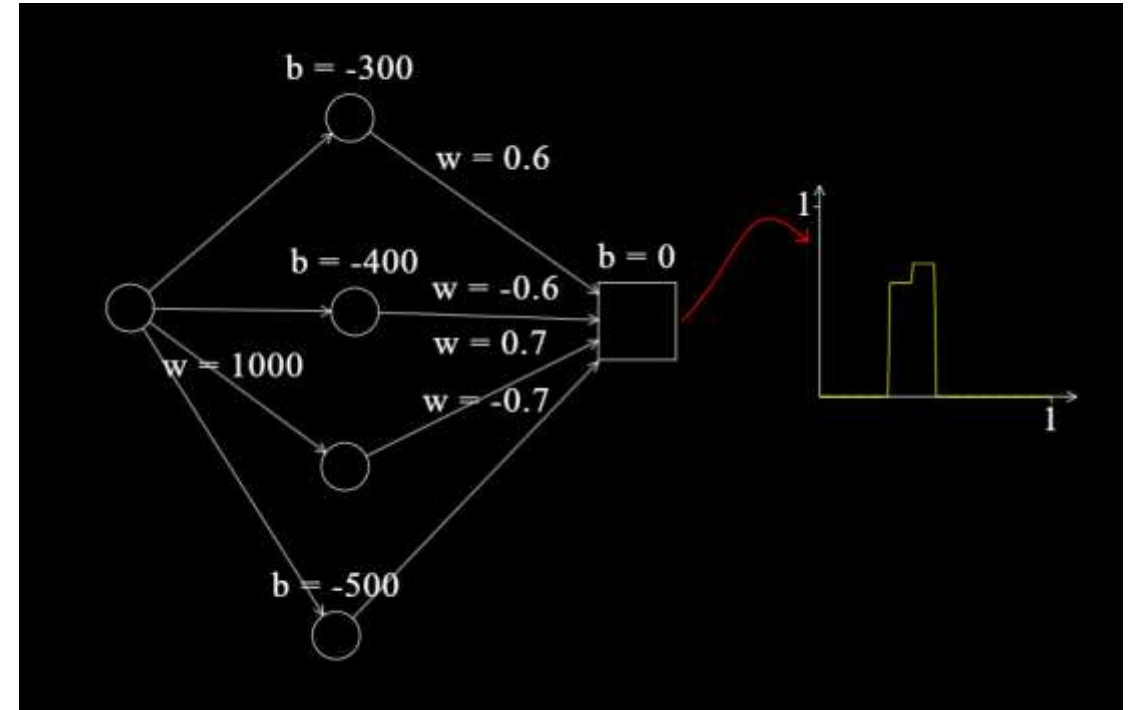
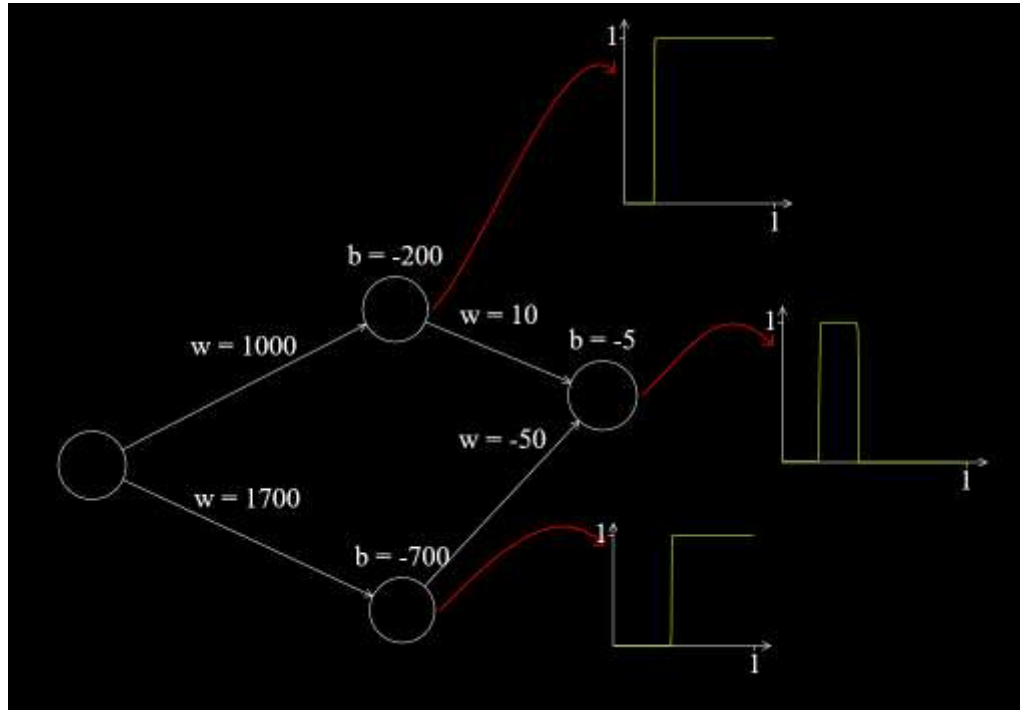


$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

1. Función Sigmoide (Logística)

- **Fórmula:** $f(x) = \frac{1}{1+e^{-x}}$
- **Rango de Salida:** $(0, 1)$

Teorema de aproximación universal



Ejercicio

Crear código en Python que resuelva la compuerta XOR, dadas la entradas AND y OR

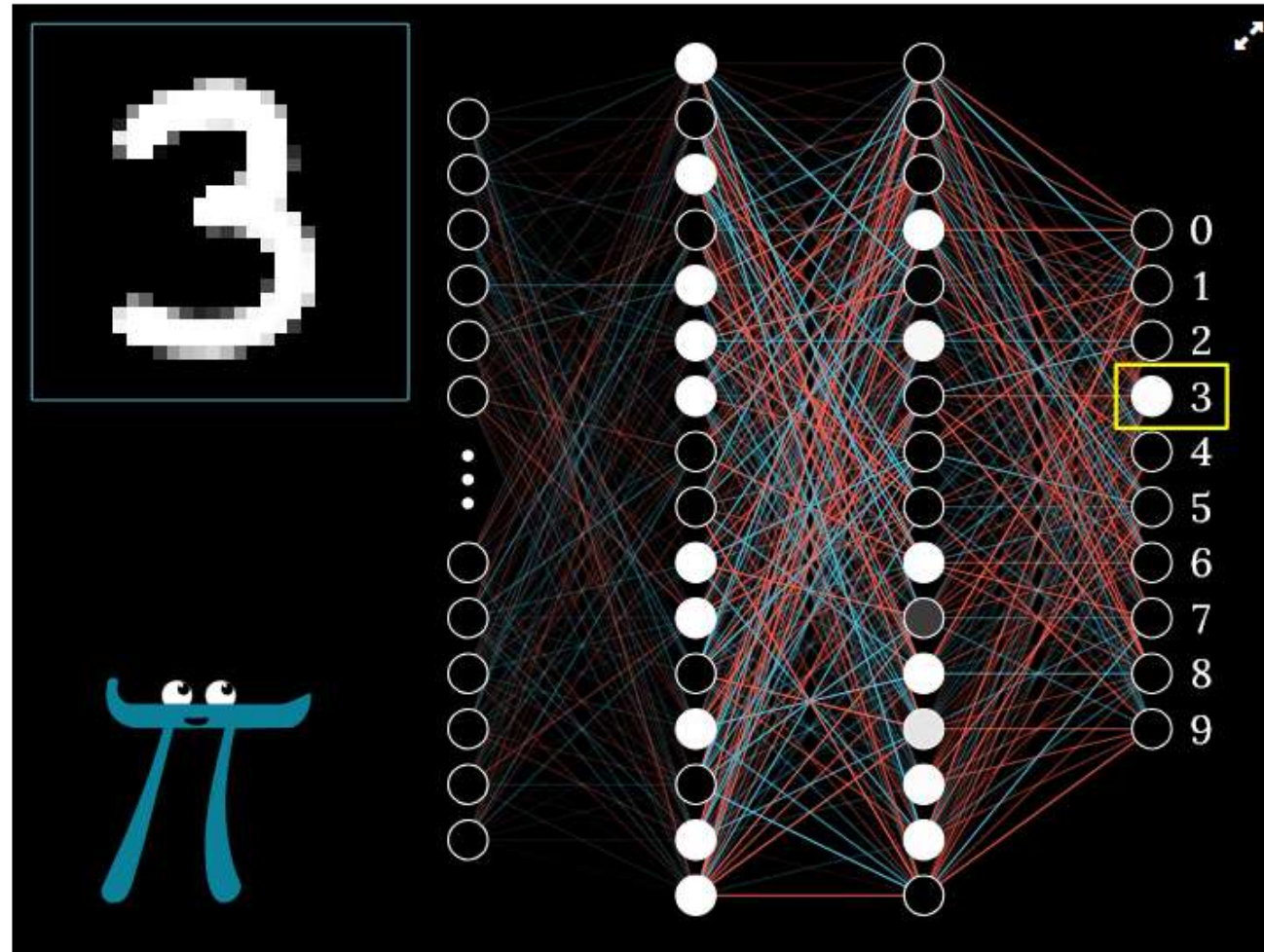
Problemas del MLP

- No existía un método efectivo para entrenar más de una capa
- Comenzó el segundo “invierno de la IA”

Red Neuronal

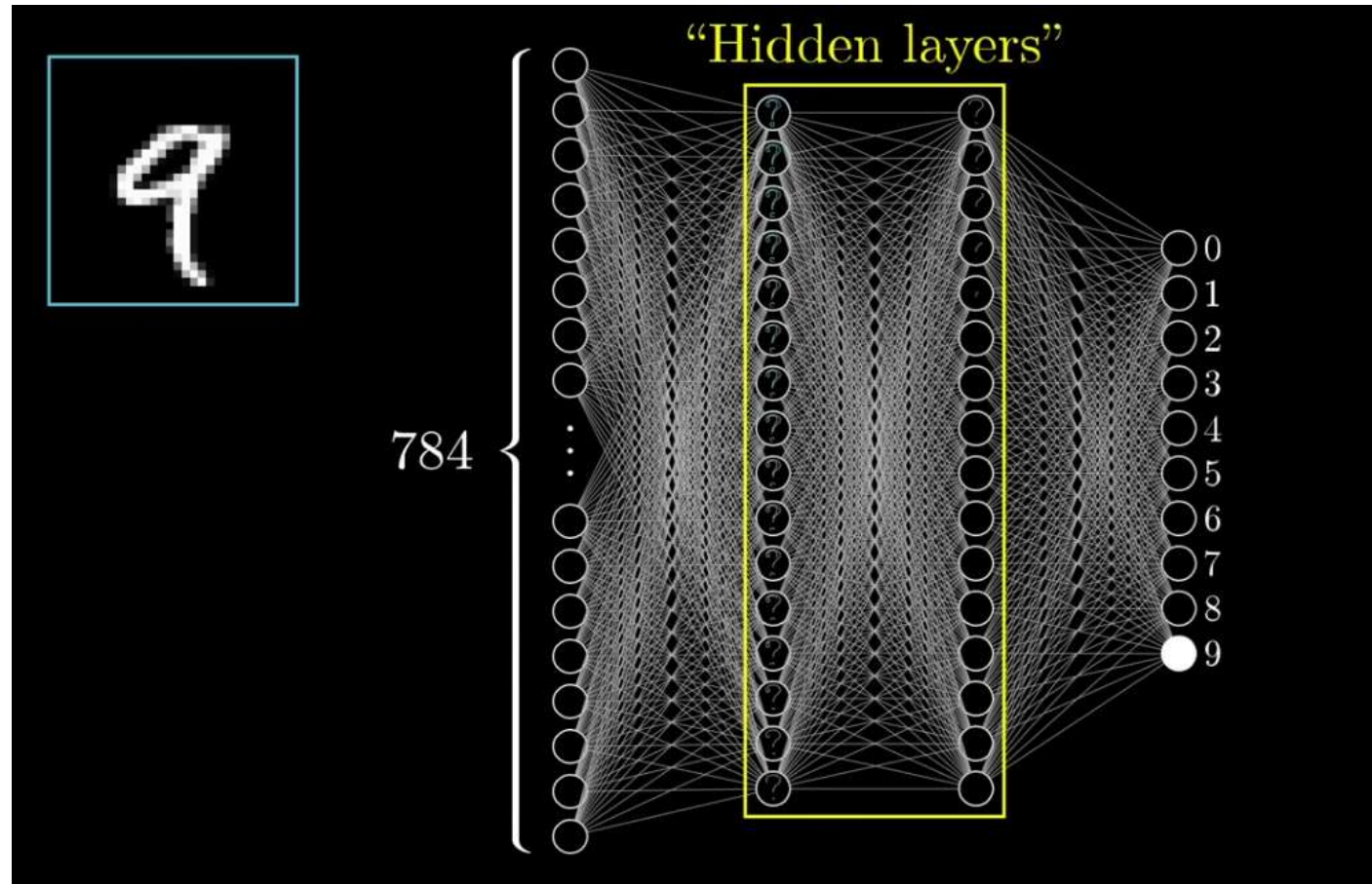
Con varias capas

Demo



<https://www.3blue1brown.com/lessons/neural-networks>

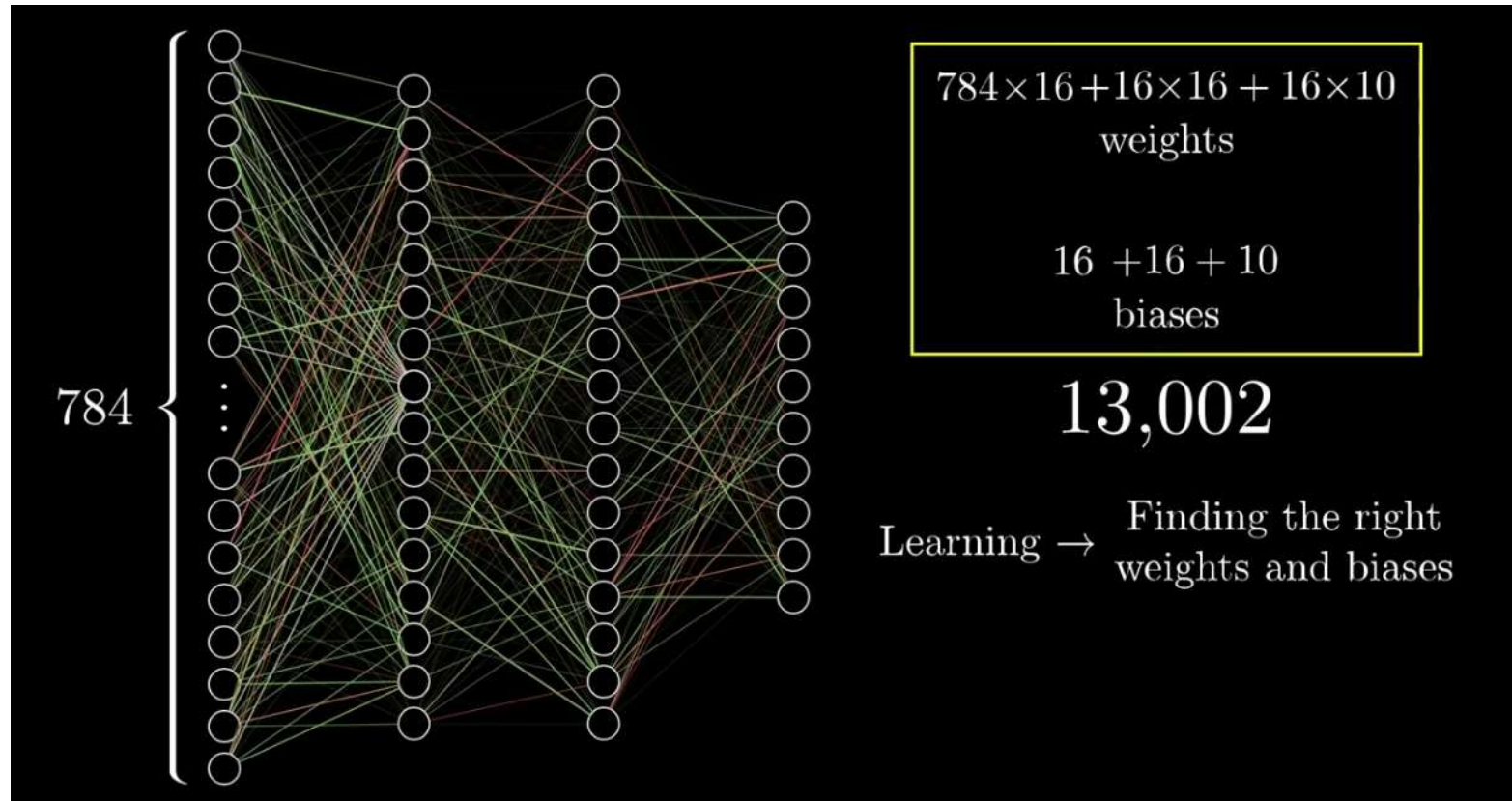
Ejemplo de Red neuronal



<https://youtu.be/aircAruvnKk?t=165> de 2:40 a 5:30

Ejemplo de Red neuronal

¿Qué es el entrenamiento?



<https://youtu.be/aircAruvnKk?t=698> de 11:39 hasta 12:42

La Retropropagación

Entrenando a las redes neuronales

La Retropropagación

1986: El MLP resurgió y se popularizó gracias a que **David Rumelhart, Geoffrey Hinton y Ronald Williams redescubrieron** y aplicaron el algoritmo de Retropropagación (Backpropagation), permitiendo por fin el entrenamiento eficiente de las redes multicapa.

La Retropropagación

Es un algoritmo esencial en el entrenamiento de redes neuronales que **calcula eficientemente los gradientes** de la **función de pérdida** (el error) con respecto a **todos los pesos y sesgos** de la red. Lo hace propagando el error desde la capa de salida hacia atrás, a través de las capas ocultas, utilizando la **regla de la cadena** del cálculo diferencial, permitiendo así determinar cómo cada ajuste individual en cualquier capa contribuye al error global y, en consecuencia, cómo deben ser modificados por un algoritmo de optimización como el **descenso del gradiente**.

Descenso del gradiente

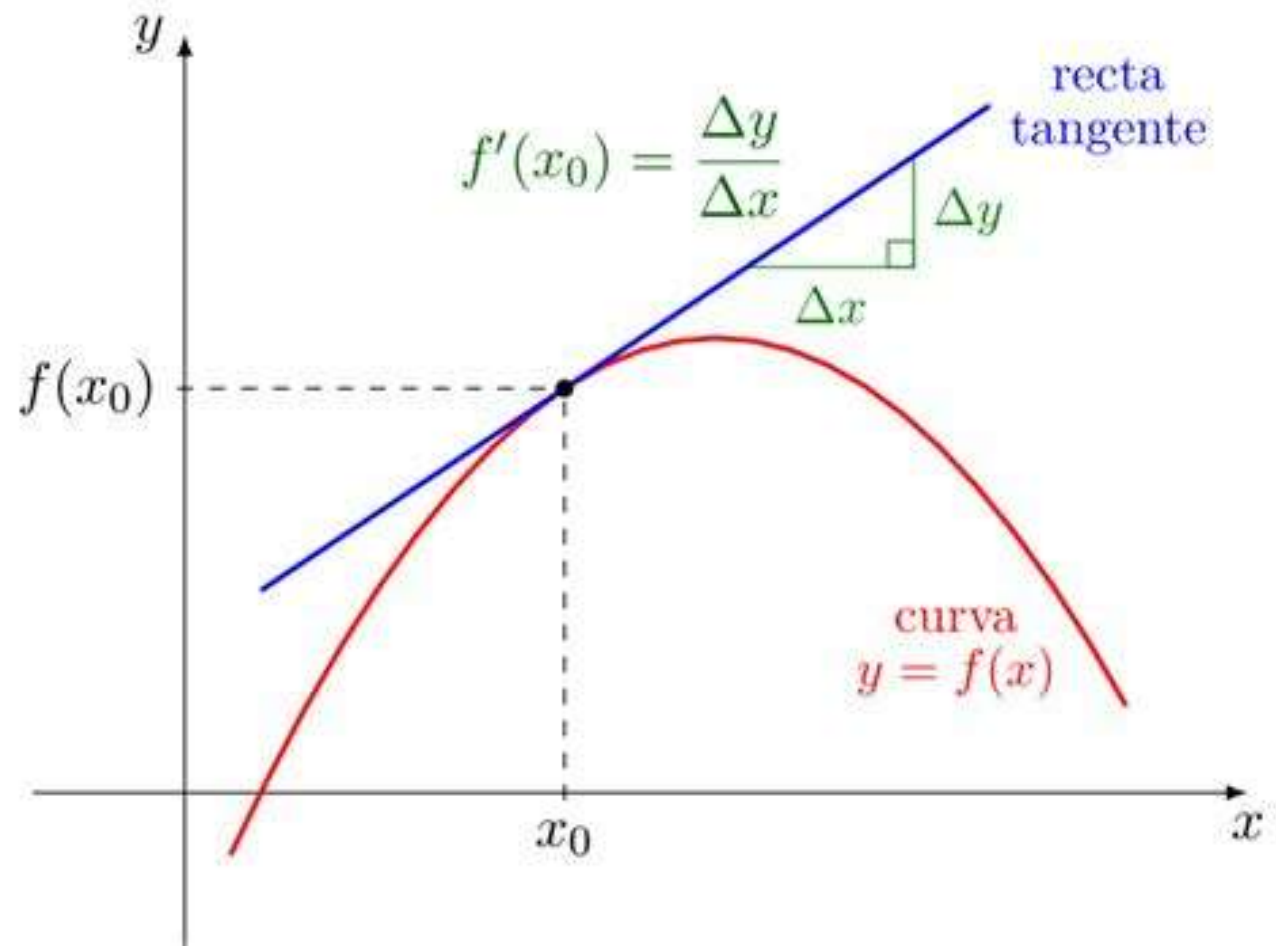
Cálculo de mínimos de funciones

Descenso del gradiente

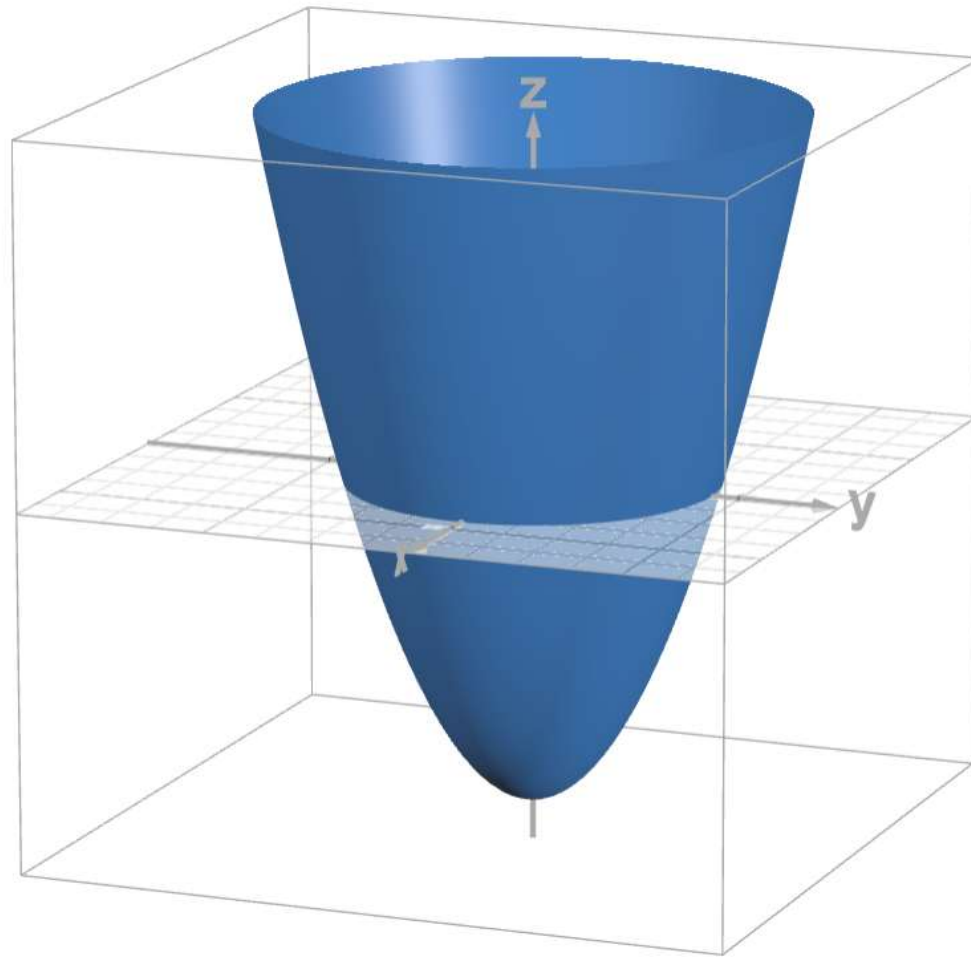
Es un algoritmo de optimización iterativo que permite **encontrar el mínimo** en una función **diferenciable**. La idea es tomar pasos de manera repetida en dirección contraria al gradiente.

La Derivada

El valor de la derivada de una función en un punto puede interpretarse geométricamente, ya que se corresponde con la pendiente de la recta **tangente** a la gráfica de la función en **dicho punto**.



Ejemplo



$$z = x^2 + y^2 - 3$$

<https://www.desmos.com/3d/qgkca5ivtn>

Descenso del gradiente en NN

Aplicado a redes neuronales permite ajustar los **pesos y sesgos** de manera gradual para **reducir el error** que comete la red al hacer predicciones.

La Función de coste

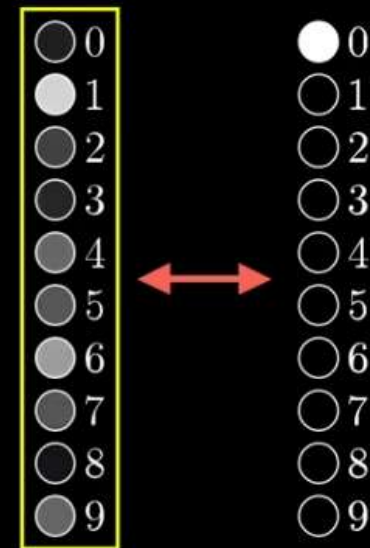
Average cost of
all training data...

Cost of



$$\left\{ \begin{array}{l} (0.12 - 1.00)^2 + \\ (0.84 - 0.00)^2 + \\ (0.26 - 0.00)^2 + \\ (0.15 - 0.00)^2 + \\ (0.40 - 0.00)^2 + \\ (0.35 - 0.00)^2 + \\ (0.62 - 0.00)^2 + \\ (0.34 - 0.00)^2 + \\ (0.08 - 0.00)^2 + \\ (0.40 - 0.00)^2 \end{array} \right.$$

What's the “cost”
of this difference?

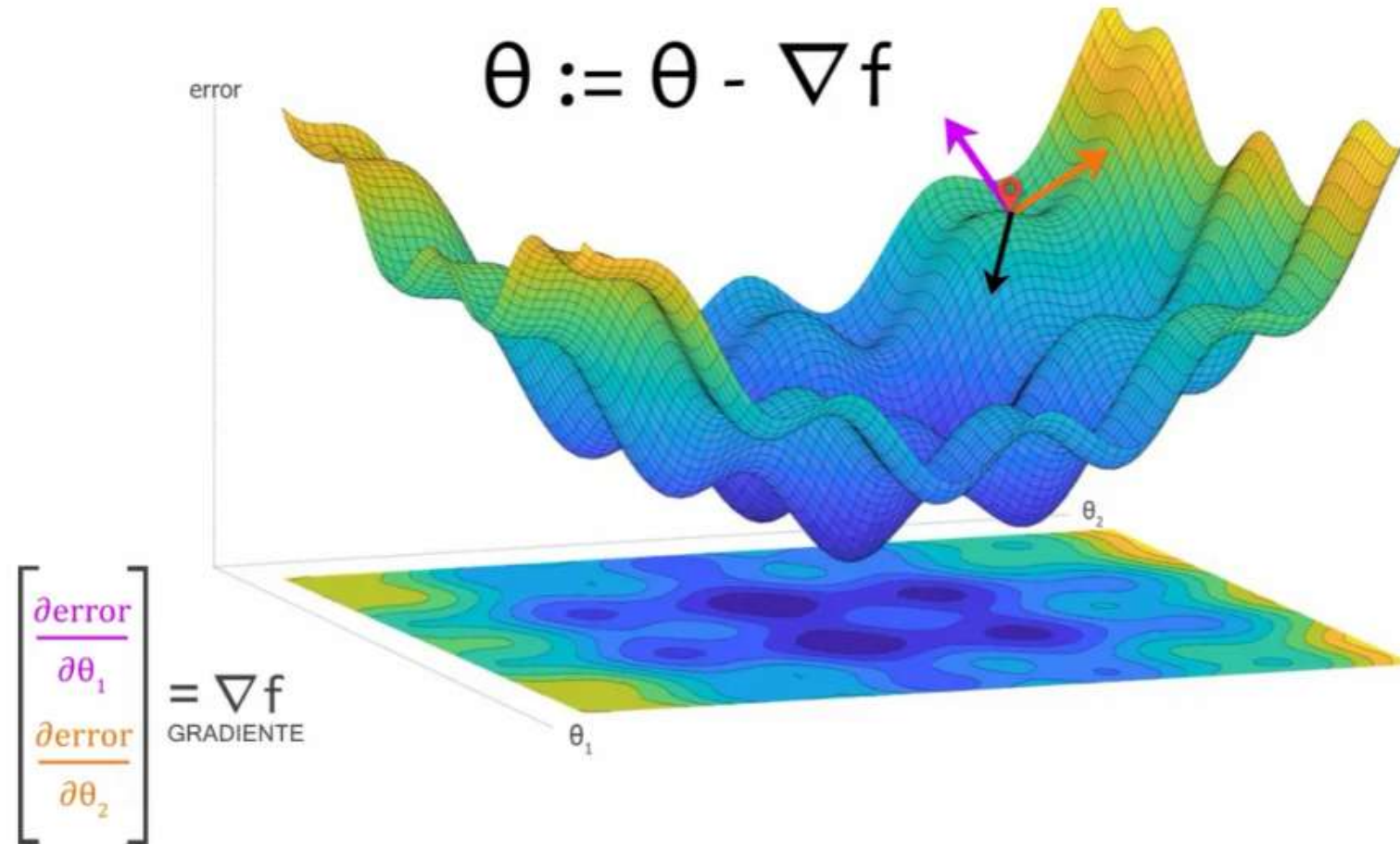


Utter trash

La Función de coste

La función de coste en el algoritmo de retropropagación es una **medida del error** entre lo que la red neuronal predice y lo que debería predecir. Su objetivo es **cuantificar qué tan mal se está desempeñando la red**. La retropropagación utiliza esta medida de error para **ajustar los pesos y sesgos de la red** de forma iterativa, buscando **minimizar ese error** y así mejorar la precisión del modelo.

Descenso del gradiente



La Retropropagación

1. Propagación Hacia Adelante (Forward Pass)

- La red toma datos de entrada, los procesa capa por capa aplicando funciones de activación, y produce una salida (predicción).

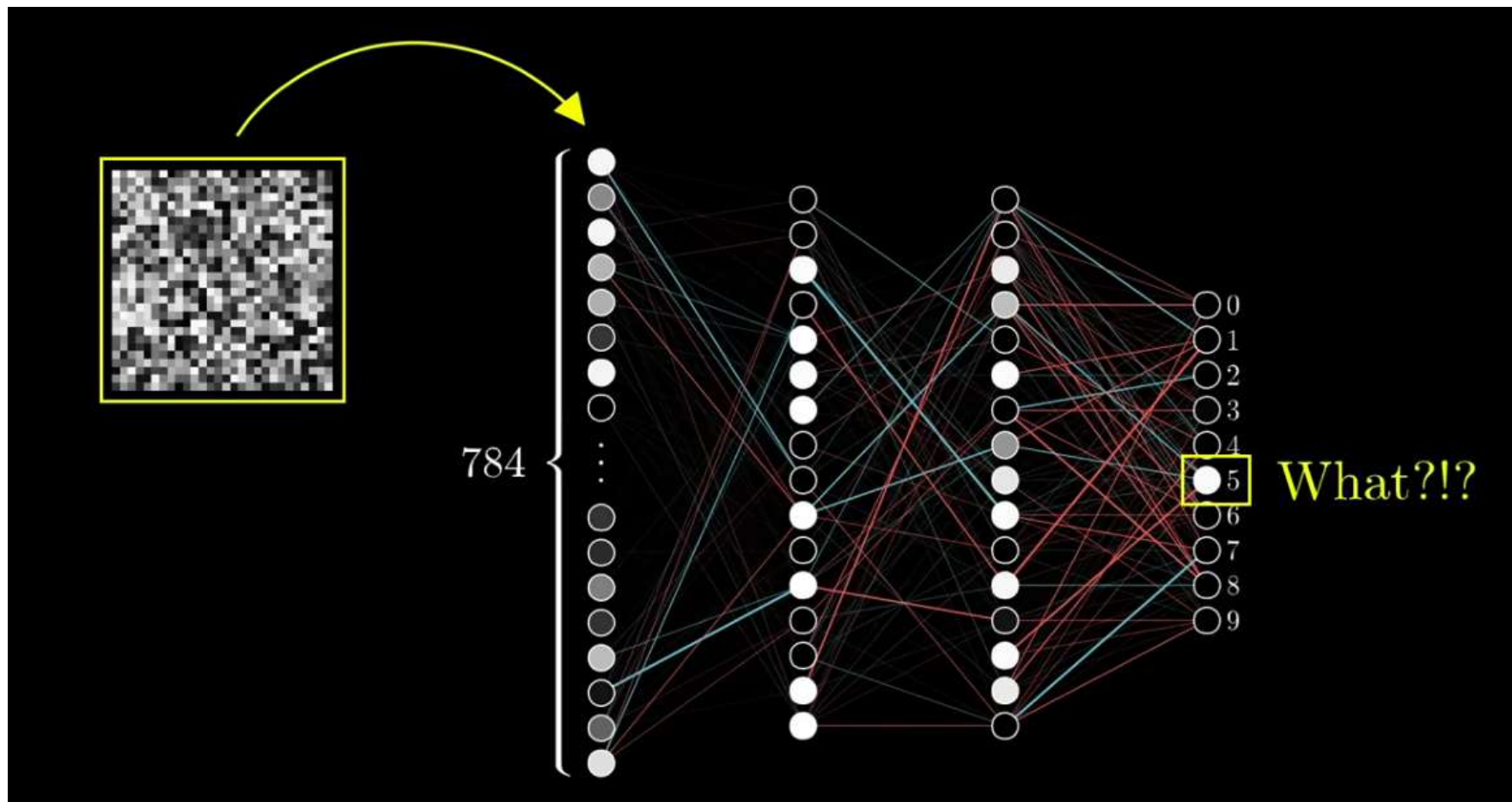
2. Propagación Hacia Atrás (Backward Pass)

- Cálculo del Error: Se compara la predicción con el valor real usando una función de pérdida para cuantificar el error.
- Cálculo de Gradientes: El error se propaga hacia atrás, calculando cómo cada peso y sesgo contribuye a ese error (usando la **regla de la cadena**).
- Actualización de Parámetros: Los pesos y sesgos se ajustan en la dirección opuesta al gradiente (mediante descenso de gradiente y una tasa de aprendizaje) para disminuir el error.

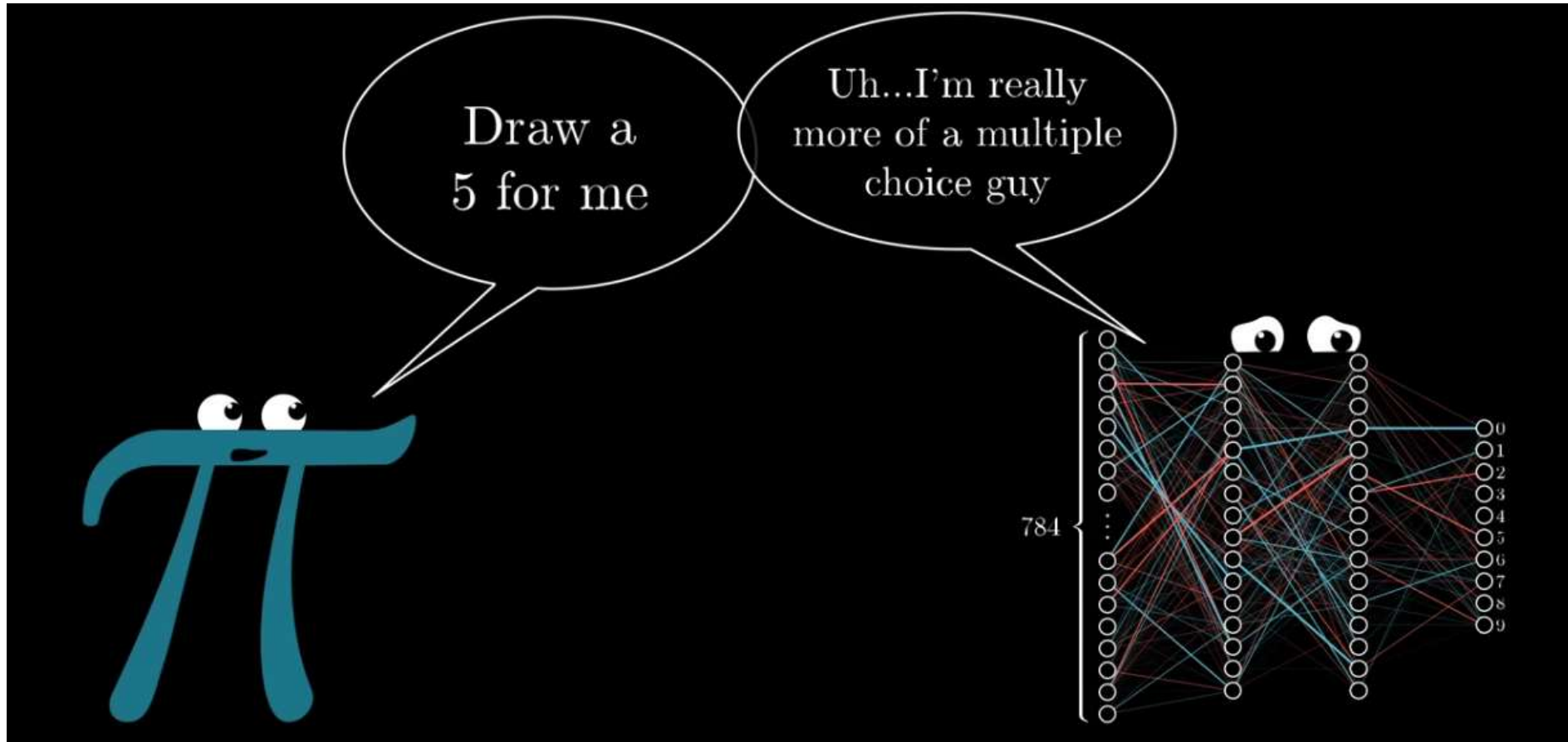
Este ciclo se repite iterativamente (en épocas) para que la red aprenda y mejore su precisión.

Problemas en las redes neuronales

Ataques



Limitaciones



Problemas en el reconocimiento de imágenes

1) Destrucción de la Estructura Espacial

- **Problema:** Un MLP requiere que la entrada sea un **vector 1D**. Para alimentar una imagen a un MLP, primero se debe **aplanar** (o vectorizar).
- **Consecuencia:** Se **pierde la información crucial de la vecindad local** (la relación espacial entre un píxel y sus vecinos inmediatos). Esta relación es esencial para identificar características visuales como bordes, líneas y texturas. El MLP trata a cada píxel como una característica independiente y desordenada.

2) Explosión de Parámetros

- **Problema:** Un MLP utiliza **conexión total** (*fully connected*).
- **Consecuencia:** El número de parámetros (pesos) crece exponencialmente, especialmente con imágenes de alta resolución. Ejemplo: Una imagen pequeña de 100 x 100 x 3 (color) tiene 30.000 píxeles de entrada. Si la primera capa oculta tiene solo 1.000 neuronas, ya hay 30 millones de pesos que aprender solo en esa capa.

Problemas en el reconocimiento de imágenes

3) Ausencia de Invariancia Traslacional

- **Problema:** Los pesos en un MLP están ligados a posiciones específicas en el vector de entrada.
- **Consecuencia:** Si la red aprende a reconocer un objeto (ej. un círculo) en la esquina superior izquierda, tendrá que aprenderlo de nuevo con un conjunto de pesos completamente diferente si el círculo aparece en la esquina inferior derecha.

Ejemplo

- Abrir el archivo Red neuronal.ipynb
- Hacer cambios en los hiper-parámetros de la red para mejorar la precisión.
 - Tasa de Aprendizaje
 - Número de Épocas
 - Tamaño del Lote
- Modernización de Componentes Clave
 - Optimizador: Reemplace el optimizador SGD por alternativas modernas como Adam o RMSprop.
 - Activación: Cambie la función de activación de las capas ocultas de Sigmoidal a ReLU
 - Función de Costo: (Si usó MSE inicialmente) Cambie a la Entropía Cruzada Categórica.

Ejemplo

Modificaciones Arquitectónicas

- Profundidad y Ancho: Experimente con la topología de la red. Pruebe añadiendo una o dos capas ocultas adicionales, o ajustando el número de neuronas en cada capa (ej. de 16 a 64).

Ejercicios

- Clasificación Binaria: clasificar solo si la imagen es el dígito “5” o “no es el 5”
- Detección par/impar: Entrena la red para clasificar si un dígito es par o impar.
- Detección de Números Grandes/Pequeños: Problema: Clasifica los dígitos en dos grupos: "pequeños" (0, 1, 2, 3, 4) o "grandes" (5, 6, 7, 8, 9).

Qué es un Tensor

Es una **generalización** de los conceptos de **escalar** (tensor de orden 0), **vector** (tensor de orden 1) y **matriz** (tensor de orden 2) a **cualquier número de dimensiones u orden** (*rank*).

	Scalars	Vectors	Matrices		
	0th order tensor rank-0 tensor	1st order tensor rank-1 tensor	2nd order tensor rank-2 tensor	3rd order tensor rank-3 tensor	4th order tensor rank-4 tensor
	s	$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$	$\begin{bmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{bmatrix}$	$\begin{bmatrix} T_{111} & T_{112} & T_{121} & T_{122} \\ T_{211} & T_{212} & T_{221} & T_{222} \end{bmatrix}$	?
index notation	s	v_i $i \in \{1, 2\}$	F_{ij} $i, j \in \{1, 2\}$	T_{ijk} $i, j, k \in \{1, 2\}$	C_{ijkl} $i, j, k, l \in \{1, 2\}$

CNN

Redes Neuronales Convolucionales

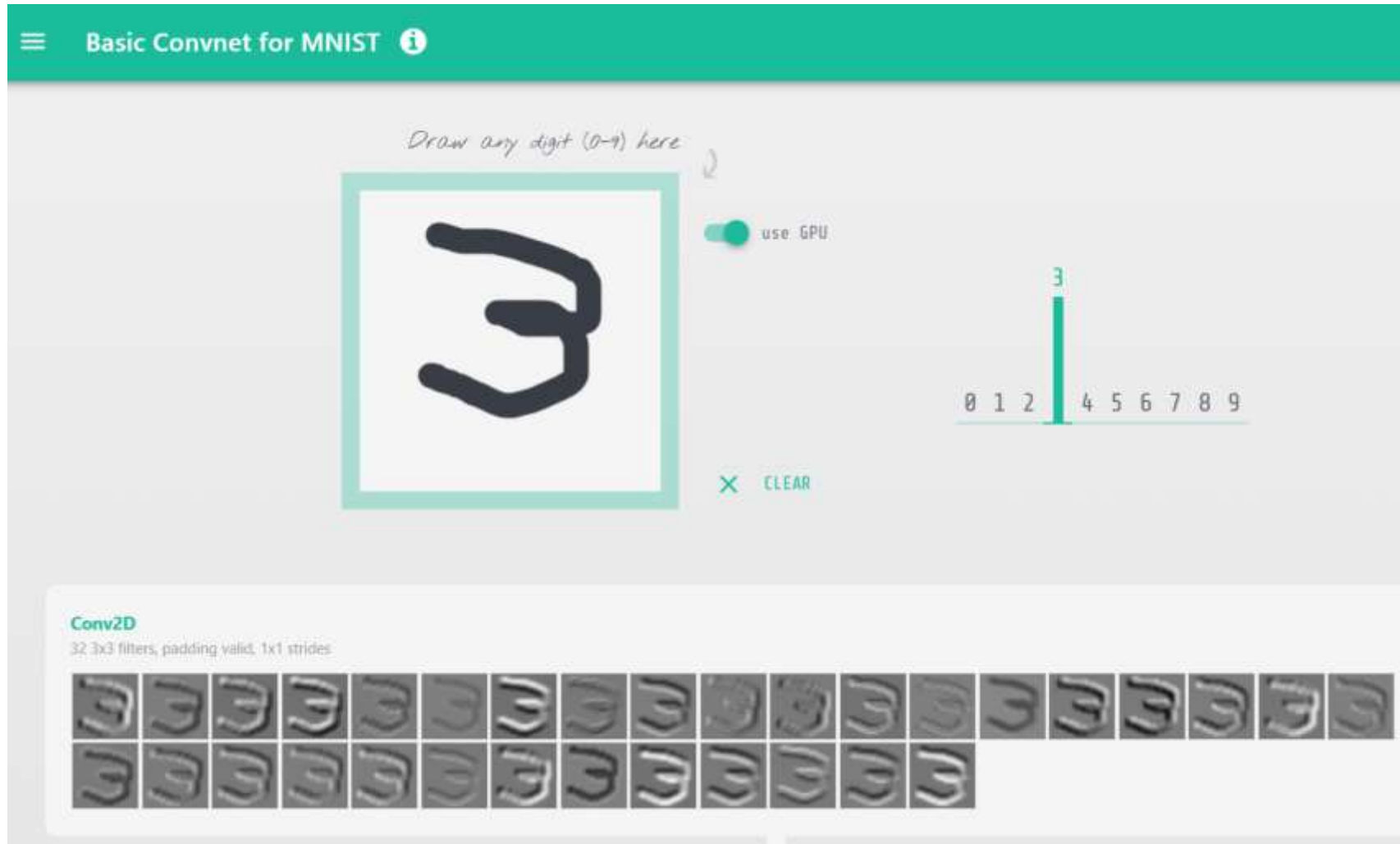
Temario

- Definición
- Demo
- Ventajas de una CNN
- Elementos distintivos
- Arquitectura
- Funcionamiento de cada parte
 - Kernel
 - Convolución
 - Función de activación
- Ejemplo
- Ejercicio

Definición

Una Red Neuronal Convolutacional (CNN), del inglés *Convolutional Neural Network*, es un tipo de **red neuronal artificial** de aprendizaje profundo (Deep Learning) diseñada específicamente para procesar datos que tienen una topología similar a una cuadrícula, como **imágenes**, donde las **relaciones espaciales** entre los datos son cruciales.

Demo



Ventajas de una CNN

A. Compartición de Pesos

- **Mecanismo:** El **kernel**, se desliza y se aplica a toda la imagen de entrada.
- **Beneficio:** Un **único conjunto de pesos** se reutiliza para escanear miles de ubicaciones. Esto reduce drásticamente el número total de parámetros que el modelo debe aprender.

B. Extracción de Características Locales (Estructura Espacial)

- **Mecanismo:** La operación de convolución garantiza que cada neurona de la capa convolucional solo esté conectada a una **pequeña región** local de la imagen de entrada (el llamado campo receptivo).
- **Beneficio:** La red aprende primero las **relaciones entre píxeles vecinos** (características locales como bordes o líneas) y **luego las combina** en capas posteriores para formar patrones complejos (jerarquía de características).

C. Invariancia Traslacional

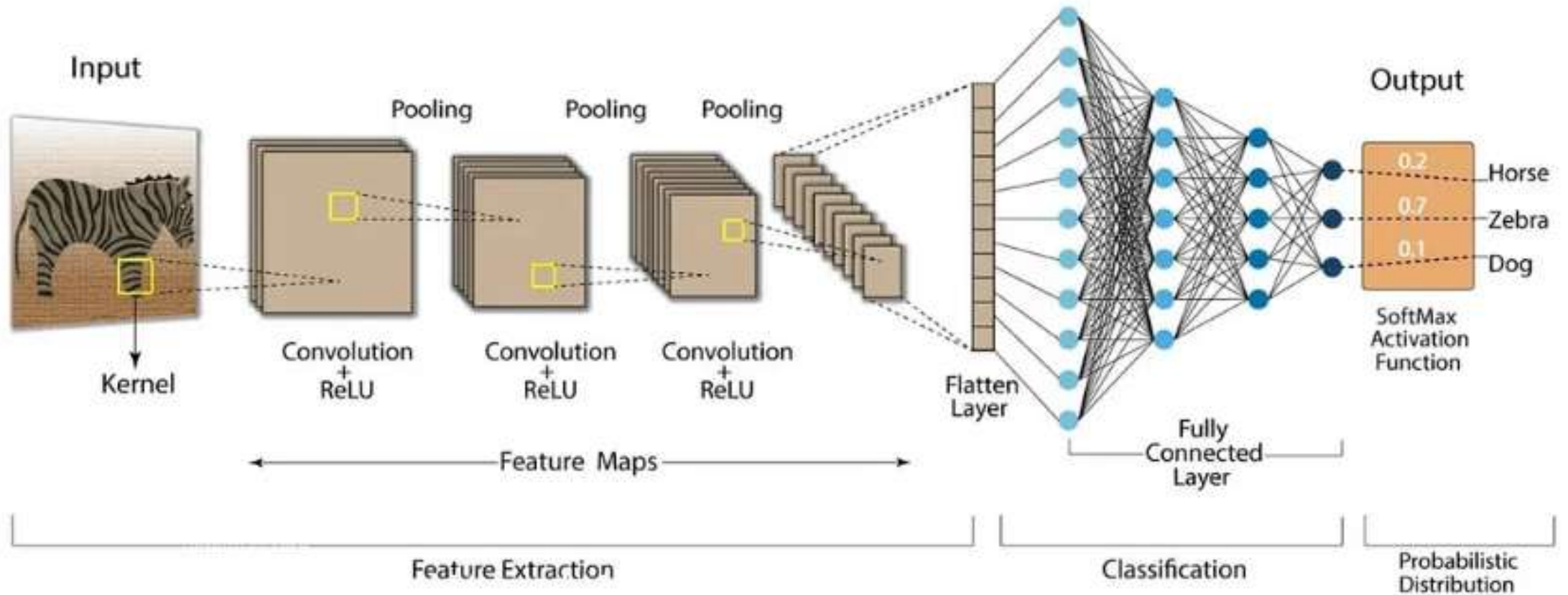
- Gracias a la compartición de pesos y a las capas de pooling, la CNN puede reconocer un patrón **independientemente** de su ubicación exacta en la imagen.

Elementos Distintivos de una CNN

A diferencia de un Perceptrón Multicapa (MLP) que requiere aplanar la entrada, una CNN **preserva la estructura espacial** de los datos mediante capas especializadas:

- Capas Convolucionales (Conv)
 - Aplican pequeños filtros llamados **kernels** que se deslizan sobre la imagen de entrada para detectar patrones locales específicos, como bordes, líneas o texturas.
- Capas de Agrupamiento (Pooling)
 - Reducir el tamaño espacial de los mapas de características generados por la capa convolucional (submuestreo). Aumenta la **invariancia traslacional** y reduce la complejidad computacional.
- Capas Totalmente Conectadas (Dense)
 - Realiza la **clasificación** final de las categorías de la imagen.

Arquitectura



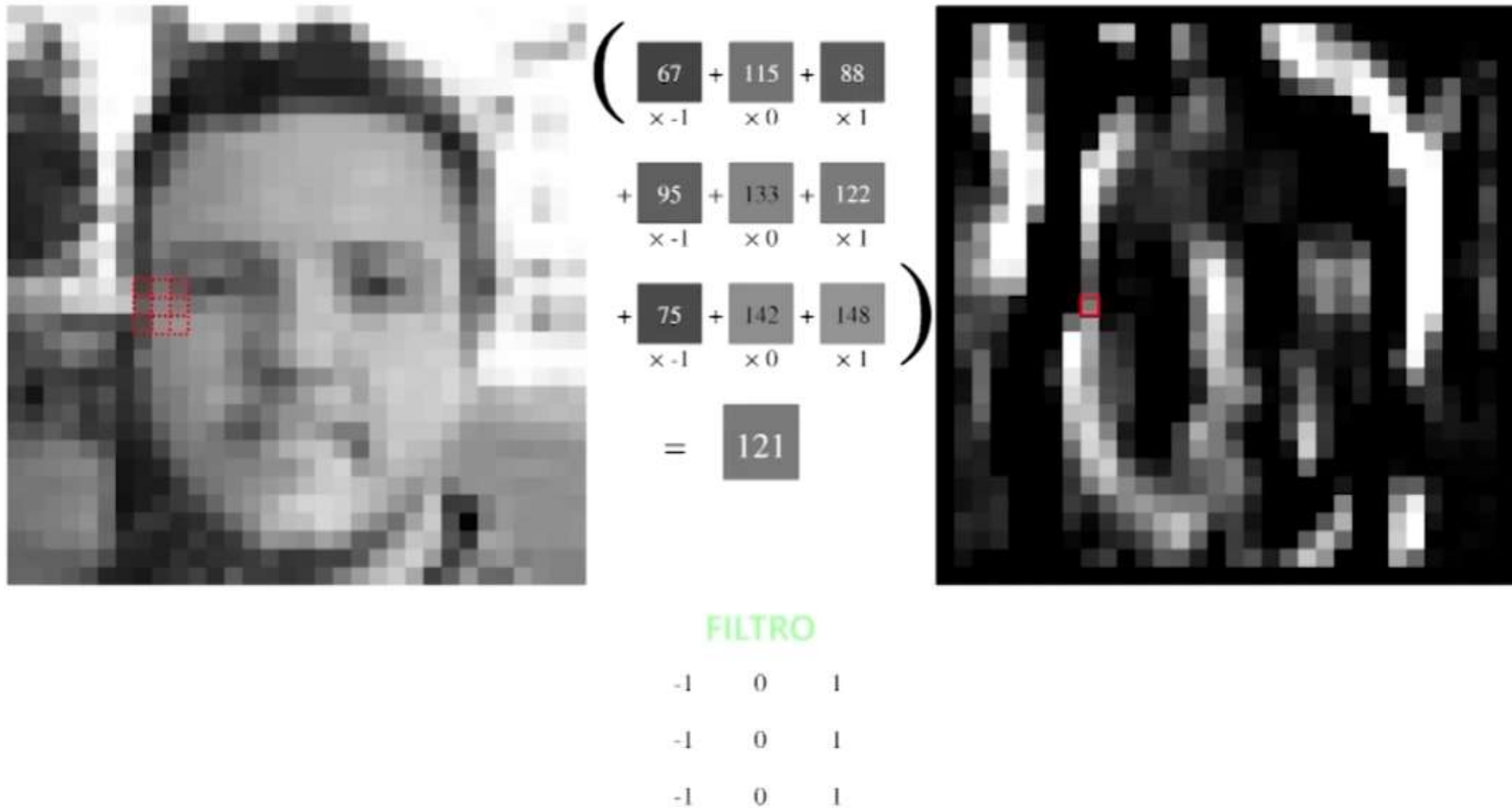
El Kernel

Es una **pequeña matriz de pesos** que se utiliza en la capa convolucional para **extraer características** específicas de la entrada (como bordes o texturas).

Funciones Clave del Kernel

- **Extractor de Características:** Actúa como un detector de patrones. Cuando los valores del kernel coinciden con los valores de un parche de la imagen, produce una **activación alta** en el mapa de características.
- **Pesos Compartidos:** El mismo kernel se aplica a **toda la imagen** mediante un desplazamiento, lo que garantiza que los patrones se reconozcan independientemente de su posición.
- **Parámetro Aprendible:** Los valores dentro de la matriz del kernel se **ajustan automáticamente** durante el entrenamiento mediante la retropropagación para minimizar el error de la red.

El Kernel



La convolución

La **convolución** es la **operación matemática** por la cual un **kernel** se **desliza** sobre la entrada (imagen o mapa de características) y calcula la **suma ponderada** de los valores de entrada cubiertos por el kernel, produciendo una única salida.

Esta operación tiene dos funciones principales:

- 1) **Extracción de Características:** Multiplica los pesos del kernel por los valores de los píxeles de la entrada y los suma, destacando cuándo el patrón del kernel es detectado.
- 2) **Generación de Mapas de Características:** El resultado de aplicar el kernel en todas las posiciones es un nuevo tensor llamado **mapa de características** (*feature map*), que representa la presencia y fuerza de ese patrón específico a lo largo de toda la entrada.

La convolución

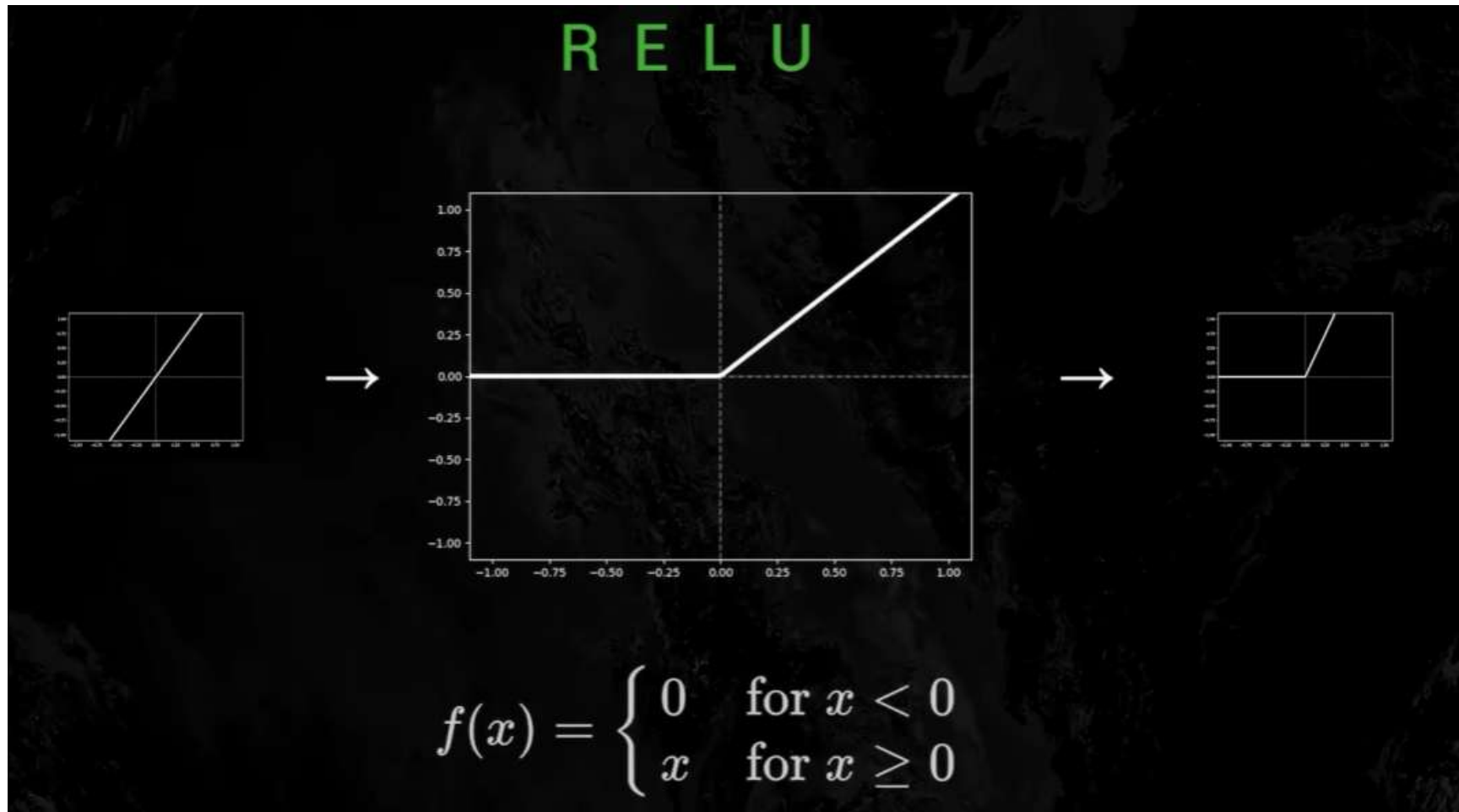


La convolución



<https://youtu.be/KuXjwB4LzSA?t=515> de 9:00 hasta 12:40

Función de activación



Ejemplo

- Abrir Keras-CNN.ipynb

Ejercicio

- Hacer cambios en la arquitectura de la red para mejorar la precisión o disminuir el tiempo que emplea.